

Lernskript Datenbanken

Klausurorientierte Wiederholung mit Erklärungen, Beispielen und Selbsttests

Erstellt auf Basis der bereitgestellten Vorlesungsunterlagen und Altklausuren

Stand: Juli 2026

Inhaltsübersicht

- [So arbeitest du mit diesem Skript](#)
- [Datenbanksysteme in einem Bild](#)
- [Konzeptuelle Modellierung mit ER-Modellen](#)
- [Vom ER-Modell zum Relationenmodell](#)
- [Relationenmodell, Schlüssel und Integrität](#)
- [Funktionale Abhängigkeiten und Normalisierung](#)
- [SQL-Datendefinition und Datenintegrität](#)
- [Relationale Algebra](#)
- [SQL-Abfragen systematisch lösen](#)
- [Sichten](#)
- [Physische Datenorganisation und Indizes](#)
- [Kompakte Prüfungssimulation](#)
- [Klausurstrategie nach Aufgabentyp](#)
- [SQL-Merkblatt erweitert](#)
- [Quellenhinweise](#)

Die Kapitel sind bewusst in der Reihenfolge angeordnet, in der die Entwurfs- und Abfragethemen aufeinander aufbauen. Für eine schnelle Wiederholung vor der Prüfung eignen sich besonders die Checklisten, Selbsttests und die kompakte Prüfungssimulation.

So arbeitest du mit diesem Skript

Dieses Skript ist **nicht als vollständiger Ersatz für die Vorlesungsfolien**, sondern als klausurorientierter Lernpfad gedacht. Viele Grundlagen werden bewusst knapp behandelt. Ausführlicher sind die Themen, die in der Datei *M_Klausurvorbereitung_ANGABE_LÖSUNG_V2.1* vorkommen und in den beiden Altklausuren besonders stark gewichtet wurden.

Die Altklausur vom 28. Juni 2021 verteilt 65 Punkte auf konzeptuelle Modellierung, relationale Abbildung, Normalisierung, Datenintegrität, DDL, relationale Algebra, SQL und Verständnisfragen. Die Klausur vom 7. Oktober 2021 setzt einen sehr ähnlichen Schwerpunkt: Modellierung und Abbildung machen dort zusammen 40 % aus; DDL, relationale Algebra und SQL weitere 45 %. Daraus folgt die sinnvolle Priorität:

1. **ER-Modellierung sicher beherrschen.**
2. **ER-Modelle fehlerfrei in Relationen übersetzen.**
3. **Funktionale Abhängigkeiten und Normalformen erkennen.**
4. **DDL, Integritätsbedingungen und referentielle Aktionen formulieren.**
5. **Relationale Algebra und SQL systematisch aufbauen.**
6. **Sichten, Indizes, 3-Ebenen-Architektur und Grundbegriffe als Verständnisfragen absichern.**

Empfohlene Vorgehensweise: Lies einen Abschnitt, rechne den Selbsttest ohne in die Lösung zu schauen und notiere bei jedem Fehler die Ursache: *Notation*, *Semantik*, *Schlüssel*, *Kardinalität*, *SQL-Syntax* oder *logischer Denkfehler*. Dadurch erkennst du wiederkehrende Fehlermuster.

Klausurregel: Schreibe Annahmen explizit hin. Bei Modellierungsaufgaben werden sinnvolle Annahmen akzeptiert, solange sie zum Text passen und konsequent im Modell umgesetzt werden.

Datenbanksysteme in einem Bild

Datenbank, DBMS und Datenbanksystem

Eine **Datenbank** ist der organisierte, dauerhaft gespeicherte Datenbestand. Das **Datenbankmanagementsystem (DBMS)** ist die Software, die Speicherung, Abfrage, Änderung, Integrität, Mehrbenutzerbetrieb, Sicherheit und Wiederherstellung organisiert. Das **Datenbanksystem (DBS)** ist die Kombination aus Datenbank und DBMS.

Ein DBMS löst insbesondere Probleme, die bei bloßer Dateihaltung schnell auftreten:

- dieselbe Information wird mehrfach und widersprüchlich gespeichert,
- Struktur und Datentypen werden nicht zuverlässig kontrolliert,
- parallele Änderungen überschreiben einander,
- Anwendungen hängen direkt vom Dateiformat ab,
- Abfragen müssen in jeder Anwendung neu programmiert werden,
- Zugriffsrechte und Wiederherstellung sind schwer zentral zu verwalten.

Die wichtigsten Qualitätsziele lassen sich in drei Gruppen ordnen:

- **Integrität:** Daten sind strukturell gültig, widerspruchsfrei und auch bei parallelen Abläufen korrekt.
- **Verfügbarkeit:** Daten sind standardisiert abfragbar, dauerhaft gespeichert und unabhängig von konkreten Anwendungen nutzbar.
- **Vertraulichkeit:** Nur berechtigte Personen erhalten die vorgesehenen Zugriffe.

ACID in einer Minute

Transaktionen bündeln mehrere Datenbankoperationen zu einer logischen Einheit:

- **Atomicity:** ganz oder gar nicht.
- **Consistency:** eine Transaktion führt einen gültigen Zustand in einen gültigen Zustand über.
- **Isolation:** parallele Transaktionen wirken so, als liefen sie ausreichend getrennt.
- **Durability:** bestätigte Änderungen bleiben auch nach einem Fehler erhalten.

Für die hier betrachteten Klausuren ist meist wichtiger, **statische Integrität** über Schema, Schlüssel und Constraints zu verstehen. ACID erklärt dagegen die **dynamische Integrität** während der Ausführung.

Die 3-Ebenen-Architektur

Die ANSI/X3/SPARC-Idee trennt:

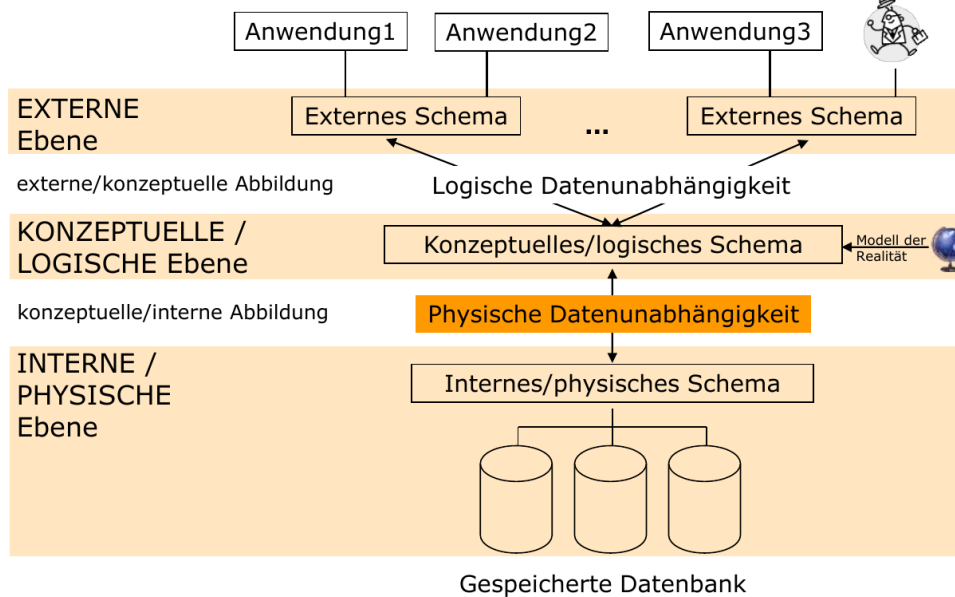
- **externe Ebene:** benutzerspezifische Sichten,
- **konzeptuelle/logische Ebene:** gesamtes logisches Datenbankschema,
- **interne/physische Ebene:** Dateien, Seiten, Indizes und Speicherstrukturen.

Logische Datenunabhängigkeit bedeutet, dass Änderungen am logischen Gesamtschema möglichst wenig Auswirkungen auf externe Sichten und Anwendungen haben. **Physische Datenunabhängigkeit** bedeutet, dass interne Speicher- oder Indexänderungen das logische Schema nicht verändern.

3-Ebenen-Architektur von Datenbanken

Physische Datenbankunabhängigkeit

ANSI/X3/SPARC: Normungsvorschlag für Systemarchitektur von DBMS



- Änderungen an der physischen Speicher- oder der Zugriffsstruktur (beispielsweise das Anlegen oder Entfernen einer Indexstruktur) hat keine Auswirkungen auf das logische/konzeptuelle Schema.

Selbsttest 1: Ein Administrator ergänzt einen Index auf `Kunde(Nachname)`. Welche Ebene ändert sich? Müssen SQL-Abfragen angepasst werden?

Lösung: Die interne/physische Ebene ändert sich. Das logische Schema und bestehende SQL-Abfragen bleiben unverändert. Genau das ist physische Datenunabhängigkeit.

Konzeptuelle Modellierung mit ER-Modellen

Vom Text zum Modell: die sichere Reihenfolge

Gehe bei jeder Modellierungsaufgabe in sechs Schritten vor:

1. **Entitätstypen markieren:** Dinge mit eigener Identität, etwa *Mitarbeiter*, *Fahrzeug*, *Buch*, *Kunde*.
2. **Schlüssel bestimmen:** Wodurch ist eine Ausprägung eindeutig? Formulierungen wie „eindeutig durch ... identifiziert“ sind direkte Hinweise.
3. **Attribute zuordnen:** Attribute gehören zu dem Konzept, dessen Eigenschaft sie beschreiben. Zeiträume und Preise gehören oft zu Beziehungen.
4. **Beziehungen benennen:** Verwende Verben oder kurze Verbgruppen: *bucht*, *gehört_zu*, *arbeitet_für*.
5. **Kardinalitäten aus jedem Blickwinkel lesen:** „Wie viele B können zu einem A gehören?“ und umgekehrt.
6. **Spezialfälle prüfen:** ternär, rekursiv, schwach, Vererbung, mehrwertiges/zusammengesetztes/abgeleitetes Attribut.

Ein häufiges Fehlermuster ist, aus jedem Substantiv eine Entität zu machen. Ein Begriff wird erst dann Entität, wenn er **eigene Identität**, **eigene Attribute** oder **eigene Beziehungen** besitzt. Ein reiner Wert wie „Gehalt“ ist meist ein Attribut; ein Gehaltsmodell mit mehreren Eigenschaften und historischer Gültigkeit kann dagegen ein eigener Entitätstyp sein.

Chen-Notation: die Bausteine

Konzept	Typische Chen-Darstellung	Bedeutung
Entitätstyp	Rechteck	Menge gleichartiger Objekte
schwacher Entitätstyp	doppeltes Rechteck	Identität hängt vom Eigentümer ab
Beziehungstyp	Raute	Menge von Beziehungen zwischen Entitäten
identifizierende Beziehung	doppelte Raute	verbindet schwache Entität mit Eigentümer
Attribut	Ellipse	Eigenschaft
Schlüsselattribut	unterstrichen	identifiziert Entität eindeutig
partieller Schlüssel	gestrichelt unterstrichen	identifiziert schwache Entität nur innerhalb des Eigentümers
mehrwertiges Attribut	doppelte Ellipse	mehrere Werte je Entität
abgeleitetes Attribut	gestrichelte Ellipse	aus anderen Werten berechenbar
Generalisierung	<i>is-a</i>	Untertyp erbt Struktur des Obertyps

Beziehungsattribute gehören an die Raute. Ein Attribut gehört dann zur Beziehung, wenn sein Wert erst durch die konkrete Kombination der beteiligten Entitäten bestimmt ist. Beispiel: *Note* gehört bei *Student prüft Vorlesung bei Professor* zur Prüfungsbeziehung, nicht pauschal zum Studenten oder zur Vorlesung.

Kardinalitäten in Chen-Notation

Bei einer binären Beziehung $A - R - B$ bedeutet die Zahl an einer Seite, wie viele Entitäten dieser Seite zu einer festen Entität der Gegenseite maximal zugeordnet sein können.

- **1:1**: Jede A-Ausprägung höchstens eine B-Ausprägung und umgekehrt.
- **1:N**: Zu einem A können viele B gehören; jedes B gehört höchstens zu einem A.
- **M:N**: Auf beiden Seiten sind mehrere Zuordnungen möglich.

Die klassische 1/N-Schreibweise enthält nur Maximalwerte. Ob Teilnahme verpflichtend ist, zeigt sie nicht zuverlässig. Dafür dient die Min-Max-Notation.

Min-Max-Notation

An jeder Kante steht $(\min, \max)$:

- $\min = 0$: Teilnahme optional,
- $\min = 1$: Teilnahme verpflichtend,
- $\max = 1$: höchstens eine Beziehungsausprägung,
- $\max = *$ oder N : beliebig viele.

Beispiel $\text{Professor } (0, *) - \text{liest} - (1, 1) \text{ Vorlesung}$:

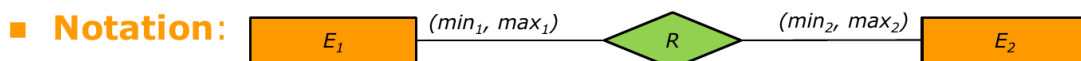
- Ein Professor kann keine, eine oder viele Vorlesungen lesen.
- Jede Vorlesung muss von genau einem Professor gelesen werden.



Datenbankentwurf

(min, max)-Notation

■ EER-Konzept: (min, max)-Notation



■ **Semantik:** Schränkt die möglichen Teilnahmen von Instanzen der beteiligten Entitätstypen an der **Beziehung** ein, indem ein minimaler und ein maximaler Wert vorgegeben wird

■ **Sonderfälle:**

- Wenn es Entities geben darf, die gar nicht an der Beziehung "teilnehmen", so ist die min-Angabe = 0
- Wenn ein Entity beliebig oft "teilnehmen" darf, wird die max-Angabe durch $*$ ersetzt
- $[0..*]$ legt somit keine Einschränkung fest

Sprachmuster richtig übersetzen

Textformulierung

„kann“
„muss“, „immer“, „genau ein“
„höchstens ein“
„mindestens ein“
„mehrere“, „beliebig viele“
„für jedes X genau ein Y“

Kardinalität

Minimum meist 0
Minimum 1
Maximum 1
Minimum 1, Maximum aus Kontext
Maximum *
X-Seite (1,1) bezogen auf Y

Selbsttest 2 - Hoteltelefone: Jedes Hotelzimmer besitzt mindestens ein Telefon. Ein Telefon ist genau einem Zimmer zugeordnet. Welche Min-Max-Angaben stehen an `Zimmer` - `besitzt` - `Telefon`?

Lösung: Beim Zimmer (1, *), beim Telefon (1, 1). Vom Zimmer aus müssen mindestens ein, eventuell mehrere Telefone erreichbar sein; jedes Telefon gehört genau zu einem Zimmer.

Normalfall: binäre Beziehungen

1:N-Beziehung

Beispiel: `Hersteller` produziert `Fahrzeugmodell`.

- Ein Hersteller produziert 0 bis viele Modelle.
- Jedes Modell gehört genau zu einem Hersteller.

Wichtig: „Modellname ist pro Hersteller eindeutig“ bedeutet nicht automatisch, dass `Modellname` weltweit eindeutig ist. Das ist ein Hinweis auf einen schwachen Entitätstyp oder einen zusammengesetzten Schlüssel.

M:N-Beziehung

Beispiel: `Student` hört `Vorlesung`.

- Ein Student kann viele Vorlesungen hören.
- Eine Vorlesung kann von vielen Studenten gehört werden.

Hat die Beziehung Attribute, zum Beispiel `Anmeldedatum`, gehören diese an die Beziehung.

1:1-Beziehung

Beispiel: `Professor` sitzt in `Raum`, falls höchstens ein Professor je Raum und höchstens ein Raum je Professor gilt.

Bei 1:1-Aufgaben entscheidet häufig die **Optionalität**, wo später der Fremdschlüssel sinnvoll liegt. Modelliert wird zunächst nur die Semantik; die relationale Umsetzung folgt in einem späteren Kapitel.

Ternäre Beziehungen

Eine ternäre Beziehung verbindet drei Entitätstypen in **einer unteilbaren Aussage**. Beispiel:

`Professor` betreut `Student` zu `Seminarthema`.

Das bedeutet nicht dasselbe wie drei getrennte binäre Beziehungen. Aus Professor betreut Student, Student bearbeitet Thema und Professor kennt Thema folgt nicht, welcher Professor welchen Studenten zu welchem Thema betreut.

Bei ternären Beziehungen liest man eine Kardinalität funktional: Fixiere die beiden anderen Entitäten. Eine 1 an Seminarthema bedeutet:

Für ein festes Paar (Professor, Student) gibt es höchstens ein Seminarthema.

Eine N an Professor bedeutet:

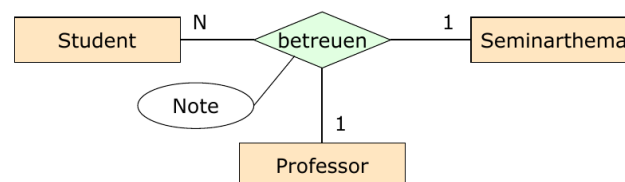
Für ein festes Paar (Student, Seminarthema) können mehrere Professoren vorkommen.



Datenbankentwurf

Kardinalitäten bei ternären (mehrstelligen) Beziehungen

- Gegeben ist die ternäre Beziehung *betreuen* zwischen den Entitätstypen *Student*, *Professor*, *Seminarthema*.



betreuen: Professor x Student → Seminarthema

betreuen: Seminarthema x Student → Professor

- Welche Integritätsbedingungen werden durch diese Kardinalitäten festgelegt?

Klausurhinweis: Zerlege eine ternäre Beziehung nur dann in binäre, wenn der Sachverhalt dies wirklich erlaubt und keine Information verloren geht. Im Zweifel bleibt sie ternär.

Selbsttest 3 - Prüfung: Ein Student legt eine bestimmte Vorlesung bei genau einem Professor ab; ein Professor kann dieselbe Vorlesung für viele Studenten prüfen; ein Student kann mehrere Vorlesungen ablegen. Modelliert man `prüfen(Student, Vorlesung, Professor)` ternär, welche funktionale Aussage drückt die 1 an Professor aus?

Lösung: (Student, Vorlesung) → Professor. Für ein festes Paar aus Student und Vorlesung ist höchstens ein Professor zuständig.

Rekursive Beziehungen

Bei einer rekursiven Beziehung nimmt derselbe Entitätstyp mehrere Rollen ein. Beispiel:

`Vorlesung` setzt `Vorlesung` voraus.

Die Rollen müssen benannt werden, etwa `Vorgänger` und `Nachfolger`. Ohne Rollennamen ist unklar, welche Seite welche Bedeutung hat.

Weitere Beispiele:

- `Mitarbeiter` ist `Vorgesetzter_von` `Mitarbeiter`,
- `Bauteil` besteht `aus` `Bauteil`,
- `Person` ist `Elternteil_von` `Person`.

Kardinalitäten beziehen sich auf Rollen, nicht auf zwei verschiedene Entitätstypen.

Schwache Entitäten

Eine schwache Entität besitzt keinen vollständigen Schlüssel aus eigenen Attributen. Ihre Identität entsteht aus:

1. dem Primärschlüssel des Eigentümers und
2. einem partiellen Schlüssel der schwachen Entität.

Beispiel: `Raum` ist nur innerhalb eines `Gebäude` durch `RaumNr` eindeutig. Der vollständige Schlüssel lautet `(GebäudeNr, RaumNr)`.

Merkmale:

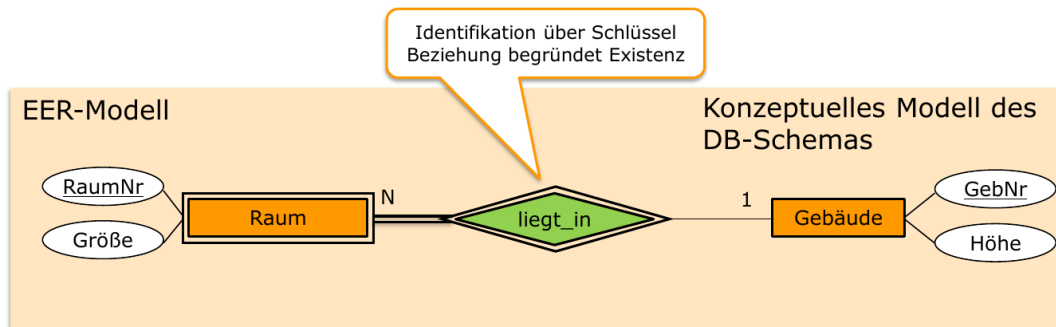
- doppeltes Rechteck,
- identifizierende Beziehung als doppelte Raute,
- vollständige Teilnahme an der identifizierenden Beziehung,
- meist 1:N vom Eigentümer zur schwachen Entität,
- partieller Schlüssel wird nur innerhalb eines Eigentümers eindeutig.

Existenzabhängige Entitätstypen 2/2

Schwache Entities

■ Beispiel EER-Modell:

- Ist die Nummer eines Raumes nur innerhalb eines Gebäudes eindeutig, so ist der Schlüssel von Räume eine Kombination aus Raum- und Gebäudenummer



Klausurbeispiel: Fahrzeugmodell

In der Klausur vom 7. Oktober 2021 ist ein Fahrzeugmodell durch `Name` und `Erscheinungsjahr` **pro Hersteller** eindeutig. Damit ist eine saubere Modellierung:

- `Hersteller(HNr, Telefon, AnsprechpartnerName)` als starke Entität,
- `Fahrzeugmodell(Name, Jahr)` als schwache Entität,
- identifizierende Beziehung `Hersteller bietet_Modell_an Fahrzeugmodell`,
- vollständiger Schlüssel des Modells: `(HNr, Name, Jahr)`.

Selbsttest 4: Ist `ExemplarNr` eines Buchexemplars ein globaler Schlüssel, wenn die Nummer nur innerhalb einer ISBN vergeben wird?

Lösung: Nein. `Buchexemplar` ist gegenüber `Buch` existenzabhängig. Vollständiger Schlüssel: `(ISBN, ExemplarNr)`.

Vererbung: Generalisierung und Spezialisierung

Bei einer `is-a`-Beziehung ist jede Untertyp-Ausprägung zugleich eine Obertyp-Ausprägung. Der Untertyp erbt:

- den Primärschlüssel,
- alle Attribute des Obertyps,
- alle Beziehungen des Obertyps.

Beispiel:

- Angestellter(PersNr, Name)
- Professor mit Rang, Raum
- Assistent mit Fachgebiet

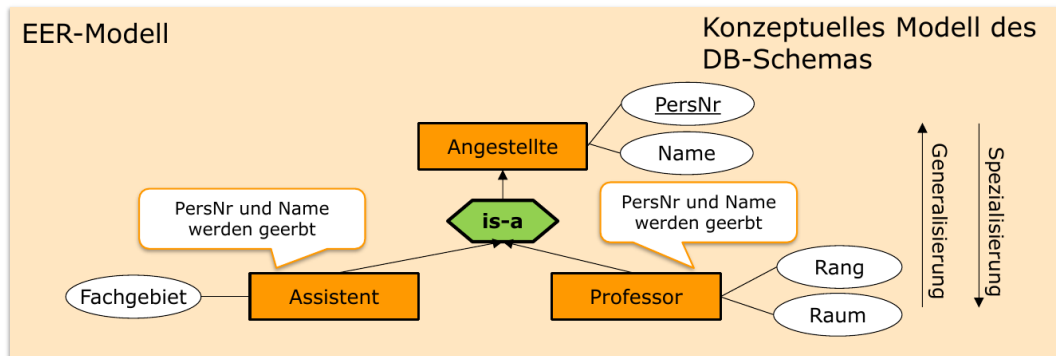
Jeder Professor ist Angestellter; nicht jeder Angestellte ist Professor.

Datenbankentwurf

Generalisierung/Spezialisierung 2/3

is-a-Beziehung mit Vererbung von Struktur

■ Beispiel EER-Modell:



© 2025

M2-63

Generalisierung/Spezialisierung mit geerbtem Schlüssel und Attributen. Quelle: M2_Datenbankentwurf, Folie 63.

Zwei zusätzliche Entscheidungen können relevant sein:

- **disjunkt vs. überlappend**: Darf eine Entität mehreren Untertypen gleichzeitig angehören?
- **vollständig vs. partiell**: Muss jede Obertyp-Ausprägung zu mindestens einem Untertyp gehören?

Wenn der Text nichts sagt, dokumentiere eine plausible Annahme.

Attribute mit besonderer Struktur

- **zusammengesetztes Attribut**: Adresse zerfällt in Straße, PLZ, Ort.
- **mehrwertiges Attribut**: mehrere Telefonnummern; in Chen doppelte Ellipse. In einem relationalen Schema wird daraus meist eine eigene Relation.
- **abgeleitetes Attribut**: Alter aus Geburtsdatum; sollte häufig nicht dauerhaft gespeichert werden.
- **Beziehungsattribut**: Preis einer Buchung, Note einer Prüfung, von/bis einer Zulagenzuordnung.

Modellierungsbeispiel aus der Altklausur: Dienstordnung

Der Sachverhalt enthält Gehaltsklassen, klassenabhängige Zulagen, Mitarbeiter und zeitabhängige Zuordnungen.

Sinnvolle Modellstruktur:

- `Gehaltsklasse(KlassenNr, Basisgehalt)`
- `Zulage(Code, Bezeichnung)` schwach unter `Gehaltsklasse`, weil der Code nur innerhalb der Klasse eindeutig ist
- `Mitarbeiter(SVNR, Vorname, Nachname)`
- `Mitarbeiter ist eingestuft_in Gehaltsklasse` mit Attribut `seit`
- `Mitarbeiter erhält Zulage` mit Attributen `von, bis`

Die letzte Beziehung ist M:N: Ein Mitarbeiter kann mehrere Zulagen erhalten; dieselbe Zulagenart kann mehreren Mitarbeitern gewährt werden. Da eine Zulage nur im Kontext einer Gehaltsklasse existiert, muss die Modellierung sicherstellen, dass ein Mitarbeiter nur Zulagen seiner Klasse erhält. Reines ER kann diese komplexe Inklusionsbedingung oft nicht vollständig ausdrücken; sie wird als zusätzliche Integritätsregel dokumentiert.

Selbsttest 5 - Modellierung unter Zeitdruck: In einem Leasingsystem wird jeder Vertrag durch `VN` identifiziert und verbindet genau einen Mitarbeiter, einen Kunden und ein Fahrzeug. Der Vertrag hat Summe, Enddatum und Rate. Welche zwei Modellierungen sind vertretbar?

Lösung: (1) `Leasingvertrag` als eigener Entitätstyp mit drei 1:N-Beziehungen zu Mitarbeiter, Kunde und Fahrzeug; das ist wegen der eigenen Identität `VN` meist am klarsten. (2) Eine ternäre Beziehung zwischen Mitarbeiter, Kunde und Fahrzeug mit `VN` und Vertragsdaten als Beziehungsattribute; dann muss `VN` als identifizierendes Merkmal der Beziehung behandelt werden. In Klausuren ist die erste Variante leichter relational abzubilden.

Typische Modellierungsfehler

1. **Kardinalitäten aus der falschen Richtung lesen.** Formuliere immer einen vollständigen Satz pro Seite.
2. **Beziehungsattribute an Entitäten hängen.** `seit`, `Preis`, `Note`, `Rolle` sind oft nur für eine Zuordnung definiert.
3. **Schwache Entität als starke modellieren.** „eindeutig pro ...“ ist ein Warnsignal.
4. **Ternäre Beziehung in drei binäre zerlegen.** Dabei geht häufig die Zuordnungskombination verloren.
5. **Untertyp mit neuem Schlüssel versehen.** Der Untertyp erbt den Schlüssel des Obertyps.
6. **Pflichtteilnahme ignorieren.** Wörter wie „muss“, „immer“ und „jedes“ beeinflussen das Minimum.
7. **Annahmen nicht dokumentieren.** Besonders bei Zeiträumen, Historisierung und Optionalität.

Vom ER-Modell zum Relationenmodell

Notation und Grundbegriffe

Ein Relationenschema wird hier als Tabellename mit Attributen notiert. Zur eindeutigen Kennzeichnung verwenden wir:

- **PK** für Primärschlüssel,
- **FK** für Fremdschlüssel,
- **PFK** für ein Attribut, das gleichzeitig Primär- und Fremdschlüssel ist.

Beispiel:

```
Professor(PersNr PK, Name, Rang, Raum)

Vorlesung(VorlNr PK, Titel, SWS, PersNr FK -> Professor.PersNr)
```

Ein Primärschlüssel muss **eindeutig**, **minimal** und **NOT NULL** sein. Ein Fremdschlüssel verweist auf einen Kandidatenschlüssel der Elterntabelle; meist ist dies der Primärschlüssel.

Starke Entitäten

Jeder starke Entitätstyp wird grundsätzlich zu einer Relation:

```
Kunde(KNr PK, Vorname, Nachname, GebDat)
```

- einfache Attribute werden Spalten,
- zusammengesetzte Attribute werden in atomare Komponenten zerlegt,
- abgeleitete Attribute werden normalerweise nicht gespeichert,
- mehrwertige Attribute werden in eine eigene Relation ausgelagert.

Beispiel mehrwertige Telefonnummer:

```
Kunde(KNr PK, Name)

KundeTelefon(KNr PFK -> Kunde.KNr, Telefonnummer PK)
```

Binäre 1:N-Beziehung

Regel: Der Primärschlüssel der 1-Seite wird als Fremdschlüssel in die N-Seite aufgenommen. Beziehungsattribute wandern ebenfalls in die N-Seite.

ER:

```
Professor 1 --- liest --- N Vorlesung
```

Relational:

```
Professor(PersNr PK, Name, Rang)

Vorlesung(VorlNr PK, Titel, SWS,
          PersNr FK -> Professor.PersNr)
```

Ist die Teilnahme der N-Seite verpflichtend (1,1), wird der Fremdschlüssel NOT NULL.

Binäre M:N-Beziehung

Regel: Erzeuge eine eigene Beziehungsrelation. Die Kombination der Fremdschlüssel ist normalerweise der Primärschlüssel. Beziehungsattribute werden dort gespeichert.

ER: Student M - hört - N Vorlesung, mit Anmeldedatum.

```
Student(Matrn PK, Name)

Vorlesung(VorlNr PK, Titel)

Hoeeren(Matrn PFK -> Student.Matrn,
         VorlNr PFK -> Vorlesung.VorlNr,
         Anmeldedatum,
         PK(Matrn, VorlNr))
```

Wenn derselbe Student dieselbe Vorlesung mehrfach zu verschiedenen Terminen hören darf, reicht (Matrn,VorlNr) nicht. Dann gehört zum Schlüssel beispielsweise Semester oder eine künstliche BelegungsNr.

Binäre 1:1-Beziehung

Drei Umsetzungen sind möglich:

1. Fremdschlüssel von A in B,
2. Fremdschlüssel von B in A,
3. separate Beziehungsrelation.

Wähle bevorzugt die Variante, die:

- keine NULL-Werte erzeugt,
- die Pflichtteilnahme direkt ausdrückt,
- wenige Relationen benötigt,
- mit UNIQUE die 1:1-Semantik absichert.

Beispiel Professor (1,1) - sitzt - (0,1) Raum: Der Fremdschlüssel RaumNr kann in Professor liegen, weil jeder Professor einen Raum hat. UNIQUE(RaumNr) verhindert, dass zwei Professoren denselben Raum erhalten.

```
CREATE TABLE Professor (
  PersNr INTEGER PRIMARY KEY,
```

```

Name    VARCHAR(50) NOT NULL,

RaumNr  INTEGER NOT NULL UNIQUE,

FOREIGN KEY (RaumNr) REFERENCES Raum(RaumNr)

);

```

Wenn beide Seiten optional sind und NULL-Werte verboten werden, ist eine separate Relation oft sauberer.

Ternäre und n-stellige Beziehungen

Eine n-stellige Beziehung wird zu einer eigenen Relation mit Fremdschlüsseln auf alle beteiligten Entitäten. Der Primärschlüssel ergibt sich aus den Kardinalitäten.

Beispiel `prüfen(Student N, Vorlesung M, Professor 1)`:

```

Pruefen(MatNr PFK -> Student.MatrNr,

        VorlNr PFK -> Vorlesung.VorlNr,

        PersNr FK  -> Professor.PersNr,

        Note,

        PK(MatNr, VorlNr))

```

Warum ist `PersNr` nicht Teil des Primärschlüssels? Die `1` an Professor bedeutet `(MatrNr, VorlNr) -> PersNr`. Das Paar aus Student und Vorlesung bestimmt den Professor bereits eindeutig.

Merksatz aus den Folien: Für den Primärschlüssel einer mehrstelligen Beziehung werden die Fremdschlüssel der Seiten benötigt, deren Kardinalität größer als 1 ist.

Rekursive Beziehungen

Bei einer 1:N-Hierarchie `Mitarbeiter` `hat_Vorgesetzten` `Mitarbeiter` genügt ein selbstreferenzierender Fremdschlüssel:

```

Mitarbeiter(MNr PK, Name,

            VorgesetzterMNr FK -> Mitarbeiter.MNr)

```

Oberste Mitarbeiter haben eventuell keinen Vorgesetzten; dann ist der FK nullable. Bei M:N-Rekursion ist eine eigene Relation nötig:

```

BauteilKomponente(

    GanzesNr PFK -> Bauteil.Nr,

    TeilNr   PFK -> Bauteil.Nr,

```

```
Menge,  
  
PK(GanzesNr, TeilNr)  
  
)
```

Rollenamen müssen in Spaltennamen sichtbar sein.

Schwache Entitäten

Für eine schwache Entität entsteht eine eigene Relation. Der Eigentümerschlüssel wird als Fremdschlüssel aufgenommen und ist Teil des zusammengesetzten Primärschlüssels.

ER: Gebäude 1 - identifiziert - N Raum, partieller Schlüssel RaumNr.

```
Gebaeude(GebNr PK, Adresse)  
  
Raum(GebNr PFK -> Gebaeude.GebNr,  
  
      RaumNr PK,  
  
      Groesse,  
  
      PK(GebNr, RaumNr))
```

Die identifizierende Beziehung benötigt keine zusätzliche Relation, sofern sie keine eigenständigen Attribute besitzt.

Vererbung relational abbilden

Das relationale Modell besitzt keine eingebaute Vererbung. In den Folien werden mehrere Varianten genannt; besonders wichtig ist das **Partitionierungsmodell** (auch „Class Table Inheritance“):

```
Angestellter(PersNr PK, Name)  
  
Professor(PersNr PFK -> Angestellter.PersNr, Rang, Raum)  
  
Assistent(PersNr PFK -> Angestellter.PersNr, Fachgebiet)
```

Vorteile: keine NULL-Spalten für Untertypattribute, gemeinsame Attribute nur einmal. Nachteil: Für vollständige Untertypobjekte ist ein Join nötig.

Weitere Varianten:

- **Einrelationenmodell / Table per Hierarchy:** eine Tabelle mit Typattribut und nullable Untertypspalten.
- **Volle Redundanz / Table per Concrete Class:** jede konkrete Untertyp-Tabelle enthält auch geerbte Attribute; gemeinsame Daten werden dupliziert.
- **Basisrelationenmodell:** nur Obertyp- und Beziehungssicht, Untertypmerkmale werden je nach Kursnotation separat behandelt.

In den Altklausuren ist die sichere Standardlösung meist das Partitionierungsmodell.

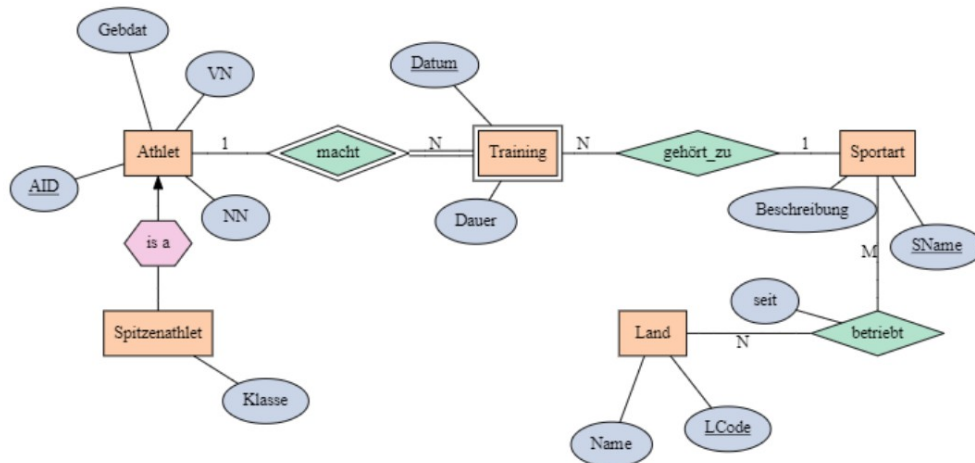
Beispiel aus KLA1: Athlet, Training, Sportart, Land

ellen Sie ein relationales Modell, das Sie aus dem folgenden ER-Diagramm ableiten. Achten Sie hierbei auf die *korrekte Transformation* von **schwachen Entitäten** und **Vererbungsbeziehungen** sowie auf Erstellen von **korrekten Fremdschlüsseln**.

Verwenden Sie dazu folgende Notation wobei PK für Primärschlüssel-, FK für Fremdschlüssel- sowie FPK für kombinierte Fremd- und Primärschlüssel-Attribute steht.

PK, A2, R2.FK, R3.FPK)

Datentypen müssen *nicht* angegeben werden



ER-Ausschnitt der Klausur vom 7. Oktober 2021 mit schwacher Entität und Vererbung.

Eine passende relationale Abbildung ist:

```
Athlet(AID PK, NN, VN, GebDat)
```

```
Spitzenathlet(AID FPK -> Athlet.AID, Klasse)
```

```
Training(AID FPK -> Athlet.AID,
```

```
    Datum PK,
```

```
    Dauer,
```

```
    PK(AID, Datum))
```

```
Sportart(SName PK, Beschreibung)
```

```
TrainingSportart(
```

```
    AID FPK -> Training.AID,
```

```
    Datum FPK -> Training.Datum,
```

```
    SName FK -> Sportart.SName,
```

```
    PK(AID, Datum))
```

Land(LCode PK, Name)

Betreibt(SName PFK -> Sportart.SName,
LCode PFK -> Land.LCode,
seit,
PK(SName, LCode))

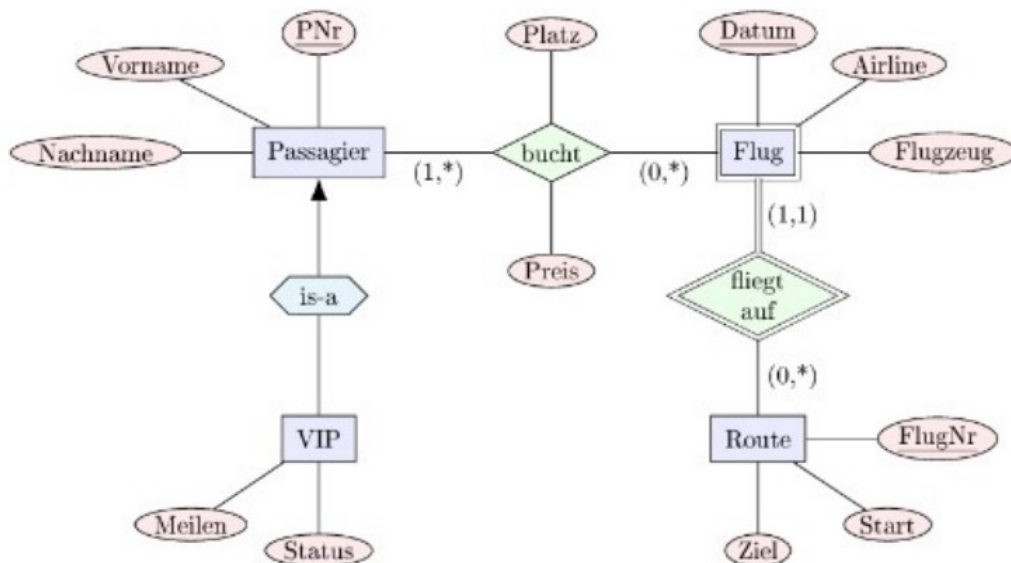
Begründung:

- Training ist schwach gegenüber Athlet; Datum ist nur innerhalb eines Athleten eindeutig.
- Spitzenathlet erbt AID und verwendet ihn als PK/FK.
- Training gehört zu Sportart ist N:1; der FK SName könnte alternativ direkt in Training stehen. Oben wurde die separate Relation nur zur sichtbaren Trennung gezeigt. Minimaler wäre:

Training(AID PFK, Datum PK, Dauer, SName FK, PK(AID,Datum))

- Sportart betreibt Land ist M:N mit Attribut seit, daher eigene Relation.

Selbsttest 6 - Flugmodell: Leite die Kernrelationen aus dem folgenden Altklausurdiagramm ab. Achte auf VIP, die schwache Entität Flug und die M:N-Beziehung bucht.



ER-Diagramm aus der Altklausur vom 28. Juni 2021.

Lösungsskizze:

Passagier(PNr PK, Vorname, Nachname)

VIP(PNr PFK -> Passagier.PNr, Meilen, Status)

```

Route(FlugNr PK, Start, Ziel)

Flug(FlugNr PFK -> Route.FlugNr, Datum PK,
      Airline, Flugzeug, PK(FlugNr, Datum))

Buchung(PNr PFK -> Passagier.PNr,
        FlugNr PFK, Datum PFK,
        Platz, Preis,
        PK(PNr, FlugNr, Datum))

```

Flug ist durch Route und Datum identifiziert. **Buchung** verbindet Passagier und konkreten Flug; bei nur einem Sitz je Passagier und Flug reicht deren Kombination als Schlüssel.

NULL-Vermeidung als Entwurfsziel

Die Altklausur verlangt ausdrücklich „möglichst wenige Relationen“ und „keine Nullwerte“. Diese Ziele können konkurrieren.

Beispiel 1:1 optional auf beiden Seiten. Ein FK in einer Basistabelle erzeugt NULL-Werte für nicht teilnehmende Entitäten. Eine separate Relation vermeidet NULLs, kostet aber eine zusätzliche Tabelle.

Beispiel Vererbung: Das Einrelationenmodell ist kompakt, erzeugt aber NULLs für Attribute anderer Untertypen. Das Partitionierungsmodell verwendet mehr Tabellen, vermeidet diese NULLs.

Verlust an Modellinformation

Die Abbildung ER -> Relationenmodell ist nicht eindeutig umkehrbar. Häufig gehen explizite Modellkonzepte verloren:

- Min-Max-Kardinalitäten sind nicht vollständig aus Tabellen ablesbar,
- „schwach“ ist im Schema nur noch als zusammengesetzter PK/FK erkennbar,
- Vererbung wird simuliert,
- Beziehungen können in FKs „verschwinden“,
- manche semantischen Regeln benötigen zusätzliche Constraints oder Trigger.

Deshalb gehört zum Datenbankentwurf nicht nur das Tabellenschema, sondern auch eine Dokumentation der fachlichen Regeln.

Checkliste relationale Abbildung

1. Jede starke Entität als Relation vorhanden?
2. Jeder Schlüssel korrekt und minimal?
3. 1:N-FK auf der N-Seite?
4. M:N und n-stellige Beziehungen als eigene Relation?
5. Beziehungsattribute an der richtigen Stelle?
6. Schwache Entität: Eigentümer-PK als PFK und Teil des PK?
7. Vererbung: Untertyp-PK zugleich FK zum Obertyp?

8. 1:1 mit `UNIQUE` abgesichert?
9. Pflichtteilnahme mit `NOT NULL` soweit möglich?
10. Mehrwertige Attribute ausgelagert?
11. Keine unnötigen NULL-Werte?
12. Alle referentiellen Wege mit FKs dokumentiert?

Relationenmodell, Schlüssel und Integrität

Relation, Tupel, Attribut und Domäne

Eine Relation ist mathematisch eine Menge von Tupeln. Daraus folgen wichtige Idealannahmen:

- keine doppelt vorkommenden Tupel,
- Reihenfolge der Tupel ist bedeutungslos,
- Reihenfolge der Attribute ist konzeptionell bedeutungslos,
- jeder Attributwert stammt aus einer Domäne.

SQL-Tabellen weichen teilweise ab: Ohne `DISTINCT` können Ergebnismengen Duplikate enthalten, und `ORDER BY` erzeugt eine explizite Reihenfolge.

Schlüsselarten

- **Superschlüssel:** Attributmenge, die jedes Tupel eindeutig identifiziert.
- **Kandidatschlüssel:** minimaler Superschlüssel.
- **Primärschlüssel:** ausgewählter Kandidatschlüssel.
- **Alternativschlüssel:** nicht gewählte Kandidatschlüssel, oft mit `UNIQUE` umgesetzt.
- **zusammengesetzter Schlüssel:** besteht aus mehreren Attributen.
- **künstlicher Schlüssel:** technisch erzeugte ID ohne fachliche Bedeutung.
- **Fremdschlüssel:** Attributmenge, die auf einen Kandidatschlüssel einer anderen oder derselben Relation verweist.

Wichtig: „Schlüssel“ bedeutet in der relationalen Theorie bereits Minimalität. Ein Superschlüssel darf überflüssige Attribute enthalten, ein Kandidatschlüssel nicht.

Vier zentrale Integritätsarten

Domänenintegrität

Jeder Wert passt zu Datentyp, Wertebereich und Format. Umsetzung durch Datentypen, `CHECK`, `NOT NULL` und gegebenenfalls benutzerdefinierte Typen.

Entitätsintegrität

Primärschlüsselwerte sind eindeutig und nicht NULL. Kein reales Objekt darf versehentlich mehrfach unter demselben Schlüssel auftreten.

Referentielle Integrität

Jeder nicht-NULL-Fremdschlüsselwert muss in der referenzierten Elterntabelle existieren.

Benutzerdefinierte Integrität

Fachliche Regeln, etwa:

- Enddatum $\geq$ Startdatum,
- Betrag > 0 ,
- Regalbestand $\leq$ Lagerbestand,
- eine Zulage muss zur Gehaltsklasse des Mitarbeiters gehören.

Referentielle Integrität



- Referentielle Integritätsbedingungen entstehen zwischen **Primär-** und **Fremdschlüssel**.

Beispiel

```

CREATE TABLE Professor(
  PersNr    INTEGER PRIMARY KEY,
  Name      VARCHAR(30) NOT NULL,
  ...);

CREATE TABLE Vorlesung(
  VorlNr    INTEGER PRIMARY KEY,
  Titel     VARCHAR(30),
  SWS       INTEGER,
  gelesenVon INTEGER,
  CONSTRAINT FK_gelesenVon
    FOREIGN KEY (gelesenVon)
    REFERENCES Professor(PersNr)
);
  
```

Vatertabelle (Professor) and **Kindtabelle** (Vorlesung) are indicated by brackets. The foreign key constraint `FK_gelesenVon` links the `gelesenVon` attribute in `Vorlesung` to the `PersNr` attribute in `Professor`.

- Zu jedem **Fremdschlüsselwert** der referenzierenden Kindtabelle (Vorlesung) muss ein entsprechender **Schlüsselwert** in der referenzierten Vatertabelle (Professor) existieren.

Integritätsprüfung an einer Datenlage

Die Altklausur verwendet Professor und Vorlesung mit:

```

Professor(
  PersNr PRIMARY KEY,
  Name NOT NULL,
  Rang CHECK (Rang IN ('C3', 'C4')),
  Raum UNIQUE
)

Vorlesung(
  VorlNr PRIMARY KEY,
  
```

```
Titel,  
SWS,  
gelesenVon FOREIGN KEY REFERENCES Professor(PersNr)  
)
```

Prüfe Statements immer in dieser Reihenfolge:

1. Syntax und Spaltenanzahl,
2. NOT NULL,
3. Datentyp/Domäne/CHECK,
4. Primärschlüssel-Eindeutigkeit,
5. UNIQUE,
6. Fremdschlüssel,
7. Auswirkungen auf Kindtabellen bei UPDATE/DELETE.

Beispiele:

- Einfügen eines Professors mit bereits vorhandener `PersNr`: verletzt PK.
- Einfügen mit neuem PK, aber bereits belegtem `Raum`: verletzt UNIQUE.
- Einfügen einer Vorlesung mit existierender `gelesenVon`: zulässig, sofern `VorlNr` neu ist.
- Löschen aller Vorlesungen eines Professors: zulässig; Kinder werden gelöscht, nicht Eltern.
- Löschen eines Professors, der noch referenziert wird: ohne referentielle Aktion nicht zulässig.

Selbsttest 7: `gelesenVon` ist nullable. Darf eine Vorlesung mit `gelesenVon = NULL` eingefügt werden?

Lösung: Ja, sofern keine `NOT NULL`-Bedingung existiert. Referentielle Integrität verlangt nur für nicht-NULL-FK-Werte ein passendes Elterntupel.

Funktionale Abhängigkeiten und Normalisierung

Warum normalisieren?

Redundanz führt zu Anomalien:

- **Änderungsanomalie:** derselbe Fakt muss an vielen Stellen geändert werden.
- **Einfügeanomalie:** ein Fakt kann nicht gespeichert werden, ohne einen anderen, noch nicht existierenden Fakt einzufügen.
- **Löschanomalie:** das Löschen eines Tupels entfernt unbeabsichtigt den letzten Nachweis eines anderen Fakts.

Normalisierung zerlegt Relationen so, dass jeder fachliche Fakt möglichst genau einmal gespeichert wird.

Funktionale Abhängigkeit

Für Attributmengen X und Y gilt $X \rightarrow Y$, wenn in jeder zulässigen Relation zwei Tupel mit gleichen X -Werten stets auch gleiche Y -Werte besitzen.

Beispiel:

PersNr $\rightarrow$ Vorname, Nachname, PLZ

PLZ $\rightarrow$ Ort

Studiengangskennzahl $\rightarrow$ Studiengangsbezeichnung

Die Aussage ist eine **Schemaeigenschaft**, nicht nur eine Beobachtung eines aktuellen Datenbestands. Aus einer kleinen Tabelle kann man eine Abhängigkeit widerlegen, aber normalerweise nicht endgültig beweisen. Wenn zwei Zeilen denselben X -Wert und unterschiedliche Y -Werte besitzen, gilt $X \rightarrow Y$ sicher nicht.

Triviale Abhängigkeit

$X \rightarrow Y$ ist trivial, wenn Y Teilmenge von X ist, zum Beispiel $\{A, B\} \rightarrow A$.

Armstrong-Regeln und praktische Schlussregeln

Die drei Grundregeln:

1. **Reflexivität:** Wenn $Y \subseteq X$, dann $X \rightarrow Y$.
2. **Augmentation:** Aus $X \rightarrow Y$ folgt $XZ \rightarrow YZ$.
3. **Transitivität:** Aus $X \rightarrow Y$ und $Y \rightarrow Z$ folgt $X \rightarrow Z$.

Praktisch nützlich:

- **Vereinigung:** $X \rightarrow Y$ und $X \rightarrow Z$ ergibt $X \rightarrow YZ$.
- **Zerlegung:** $X \rightarrow YZ$ ergibt $X \rightarrow Y$ und $X \rightarrow Z$.

- **Pseudotransitivität:** $X \rightarrow Y$ und $WY \rightarrow Z$ ergibt $WX \rightarrow Z$.

Attributhülle und Schlüsselsuche

Die Hülle X^+ enthält alle Attribute, die aus X mit den FDs ableitbar sind.

Algorithmus:

1. Starte mit $X^+ = X$.
2. Für jede FD $A \rightarrow B$: Wenn $A \subseteq X^+$, füge B zu X^+ hinzu.
3. Wiederhole, bis nichts mehr wächst.
4. Ist X^+ die gesamte Relation, ist X ein Superschlüssel.
5. Prüfe Minimalität, indem jedes Attribut testweise entfernt wird.

Beispiel

Relation $R(A, B, C, D, E)$ mit:

```
A -> B
B C -> D
D -> E
```

$(A, C)^+$:

- Start $\{A, C\}$
- aus $A \rightarrow B$: $\{A, B, C\}$
- aus $BC \rightarrow D$: $\{A, B, C, D\}$
- aus $D \rightarrow E$: $\{A, B, C, D, E\}$

AC ist Superschlüssel. Weder A^+ noch C^+ umfasst alle Attribute, daher ist AC Kandidatenschlüssel.

Abhängigkeiten in einer konkreten Tabelle prüfen

Aus KLA1 stammt die Tabelle:

A	B	C	D
1	8	8	9
2	7	2	8
4	6	3	9
2	7	4	2
1	4	2	9
9	4	2	9

Prüfung:

- $C \rightarrow D$ gilt **nicht**: $C=2$ führt zu $D=8,2,9$.
- $B \rightarrow A$ gilt **nicht**: $B=4$ führt zu $A=1$ und $A=9$.
- $BC \rightarrow D$ gilt im gezeigten Bestand: wiederholte BC -Kombinationen liefern keinen Widerspruch.
- $D \rightarrow A$ gilt **nicht**: $D=9$ gehört zu $A=1,4,9$.
- $A \rightarrow B$ gilt **nicht**: $A=1$ führt zu $B=8$ und $B=4$.
- $B \rightarrow C$ gilt **nicht**: $B=7$ führt zu $C=2$ und $C=4$; $B=4$ führt zu $C=2$ in beiden Zeilen, aber ein

Gegenbeispiel reicht.

Achtung: „gilt im Datenbestand“ ist schwächer als „gilt fachlich im Schema“.

Erste Normalform (1NF)

Eine Relation ist in 1NF, wenn jede Zelle einen atomaren Wert der vorgesehenen Domäne enthält. Nicht 1NF sind typischerweise:

- Listen in einer Spalte,
- wiederholende Spalten `Tel1`, `Tel2`, `Tel3`,
- verschachtelte Tabellenwerte im klassischen relationalen Modell.

Beispiel `Professor(PNr, Name, Tel1, Tel2, Tel3)` verletzt den Geist der 1NF durch Wiederholungsgruppen. Lösung:

```
Professor(PNr PK, Name)

ProfessorTelefon(PNr PFK, Telefon PK, PK(PNr,Telefon))
```

Zweite Normalform (2NF)

Eine Relation ist in 2NF, wenn sie in 1NF ist und jedes Nichtschlüsselattribut **voll funktional** von jedem Kandidatenschlüssel abhängt. Teilabhängigkeiten können nur bei zusammengesetzten Schlüsseln auftreten.

Beispiel:

```
Album(AlbumNr, InterpretId, Albumtitel, Erscheinungsjahr)

PK(AlbumNr, InterpretId)

AlbumNr -> Albumtitel, Erscheinungsjahr
```

Falls `AlbumNr` allein bereits Titel und Jahr bestimmt, hängen Nichtschlüsselattribute nur von einem Teil des zusammengesetzten Schlüssels ab. Dann ist die Relation nicht in 2NF.

Eine Relation mit einem einattributigen Kandidatenschlüssel ist, sofern sie in 1NF ist, automatisch in 2NF.

Dritte Normalform (3NF)

Eine gebräuchliche klausurtaugliche Formulierung:

Kein Nichtschlüsselattribut darf transitiv von einem Kandidatenschlüssel über ein anderes Nichtschlüsselattribut abhängen.

Formal gilt für jede nichttriviale FD $X \rightarrow A$: Entweder X ist Superschlüssel oder A ist ein primes Attribut, also Teil eines Kandidatenschlüssels.

Beispiel:

```
Mitarbeiter(PNr, Vorname, Nachname, AbteilungsNr, Abteilungsname)

PNr -> Vorname, Nachname, AbteilungsNr

AbteilungsNr -> Abteilungsname
```

Transitiv: PNr -> AbteilungsNr -> Abteilungsname.

Zerlegung:

```
Mitarbeiter(PNr PK, Vorname, Nachname, AbteilungsNr FK)

Abteilung(AbteilungsNr PK, Abteilungsname)
```

Boyce-Codd-Normalform (BCNF)

Eine Relation ist in BCNF, wenn für jede nichttriviale FD $X \rightarrow Y$ die linke Seite X ein Superschlüssel ist.

BCNF ist strenger als 3NF. Der Unterschied tritt auf, wenn die rechte Seite ein primes Attribut ist und die linke Seite kein Superschlüssel. 3NF kann dies noch erlauben, BCNF nicht.

In typischen Klausuraufgaben reicht die Zerlegung bis 3NF; BCNF sollte dennoch begrifflich verstanden werden.

Normalisierungsschritte mit dem Professor-Studiengang-Beispiel

Aus der Klausurvorbereitung:

```
Professor_lehrt_in_Studiengang(

    Personalnr, Vorname, Nachname, PLZ, Ort,

    Studiengangskennzahl, Studiengangsbezeichnung

)
```

Semantik:

```
Personalnr -> Vorname, Nachname, PLZ

PLZ -> Ort

Studiengangskennzahl -> Studiengangsbezeichnung

(Personalnr, Studiengangskennzahl) ist Schlüssel der Lehrzuordnung
```

Zerlegung:

```
Professor(Personalnr PK, Vorname, Nachname, PLZ FK)

Ort(PLZ PK, Ort)

Studiengang(Studiengangskennzahl PK, Studiengangsbezeichnung)
```

```
Studiengang_Professor(  
    Studiengangskennzahl PFK,  
    Personalnr PFK,  
    PK(Studiengangskennzahl, Personalnr)  
)
```

HOTEL-Beispiel aus der Klausurvorbereitung

Ausgangsrelation:

```
HOTEL(BNr, NName, VName, Mail, Tel, Typ, Raum,  
      von, bis, BDatum, Bad, Betten, WC)
```

Fachliche Abhängigkeiten:

```
BNr -> BDatum, Gast, Raum, von, bis  
GastNr -> NName, VName, Mail, Tel  
Raum -> Typ  
Typ -> Bad, Betten, WC
```

Eine mögliche 3NF-Zerlegung:

```
Gast(GastNr PK, NName, VName, Mail, Tel)  
Buchung(BNr PK, BDatum, GastNr FK, RaumNr FK, von, bis)  
Zimmer(RaumNr PK, TypNr FK)  
Zimmertyp(TypNr PK, Bad, WC, Betten)
```

Wenn eine Buchung mehrere Zimmer umfassen darf:

```
Buchung(BNr PK, BDatum, GastNr FK)  
Buchung_Zimmer(BNr PFK, RaumNr PFK, von, bis,  
               PK(BNr, RaumNr))
```

Normalformen

- Gast (GastNr, NName, VName, Mail, Tel)
- Buchung (BNr, BDatum, GastNr, RaumNr, von, bis)
- Zimmer (RaumNr, TypNr)
- Zimmertyp (TypNr, Bad, WC, Betten)

Anmerkung: Obige Lösung ist akzeptabel, denn es war nicht gefordert, dass ein Gast mit einer Buchung mehrere Zimmer bestellen kann. Soll diese Anforderung berücksichtigt werden, dann ist folgende Lösung zu wählen:

- Gast (GastNr, NName, VName, Mail, Tel)
- Buchung (BNr, BDatum, GastNr)
- Buchung_Zimmer (BNr, RaumNr, von, bis)
- Zimmer (RaumNr, TypNr)
- Zimmertyp (TypNr, Bad, WC, Betten)

Altklausurbeispiel: Mautsystem

Aus dem Datensatz lassen sich fachliche Konzepte trennen:

- Fahrzeug: Kennzeichen, Typ/Modell, Farbe,
- Fahrer: FNr, Vorname, Nachname,
- Mautstelle: MNr, Bezeichnung,
- Durchfahrt: Fahrzeug, Fahrer, Mautstelle, Zeitpunkt.

Eine 3NF-Lösung ohne künstliche Schlüssel könnte sein:

Fahrzeug(Kennzeichen PK, Modell, Farbe)

Fahrer(FNr PK, Vorname, Nachname)

Mautstelle(MNr PK, MBez)

Durchfahrt(

Kennzeichen PFK -> Fahrzeug.Kennzeichen,

MNr PFK -> Mautstelle.MNr,

Zeitpunkt PK,

FNr FK -> Fahrer.FNr,

PK(Kennzeichen, MNr, Zeitpunkt)

)

Da ein Fahrzeug beliebig oft dieselbe Mautstelle passieren kann, muss der Zeitpunkt Teil des Schlüssels sein. Ob zusätzlich MNr nötig ist, hängt davon ab, ob ein Fahrzeug zum exakt gleichen Zeitpunkt an zwei Mautstellen auftreten können soll. Eine plausible Annahme ist zu dokumentieren.

Verlustfreie und abhängigkeitserhaltende Zerlegung

Eine Zerlegung soll zwei Eigenschaften möglichst erfüllen:

- **verlustfrei:** Der natürliche Join der Teilrelationen erzeugt genau die ursprünglichen zulässigen Tupel, keine Schein-Tupel.
- **abhängigkeitserhaltend:** Die ursprünglichen FDs können ohne Join durch Constraints auf den Teilrelationen geprüft werden.

Für eine binäre Zerlegung von R in R1 und R2 ist ein nützliches Kriterium: Der Schnitt $R1 \cap R2$ sollte funktional mindestens eine der beiden Teilrelationen bestimmen.

Normalform-Schnelltest

1. Gibt es Listen oder Wiederholungsspalten? -> nicht 1NF.
2. Ist der Schlüssel zusammengesetzt? Prüfe Teilabhängigkeiten -> 2NF.
3. Bestimmt ein Nichtschlüsselattribut ein anderes Nichtschlüsselattribut? -> meist nicht 3NF.
4. Gibt es eine nichttriviale FD mit Nicht-Superschlüssel links? -> nicht BCNF.

Selbsttest 8: Bestimme die höchste Normalform von `Song(AlbumNr, TrackNr, Songtitel, Albumtitel)` mit Schlüssel `(AlbumNr, TrackNr)` und `AlbumNr -> Albumtitel`.

Lösung: 1NF, aber nicht 2NF, weil `Albumtitel` nur von `AlbumNr`, also einem echten Teil des zusammengesetzten Schlüssels, abhängt.

Selbsttest 9: `Interpret(InterpretId, Name, Gründungsjahr)` mit `InterpretId` als einzigem Schlüssel und keinen weiteren FDs. Höchste Normalform?

Lösung: BCNF, damit auch 3NF, 2NF und 1NF. Jede nichttriviale FD wird vom Schlüssel bestimmt.

SQL-Datendefinition und Datenintegrität

SQL-Sprachfamilie

- **DDL:** CREATE, ALTER, DROP - Schemaobjekte definieren.
- **DML:** INSERT, UPDATE, DELETE - Daten verändern.
- **DRL/Query:** SELECT - Daten abfragen.
- **DCL/TCL:** Rechte und Transaktionen, etwa GRANT, REVOKE, COMMIT, ROLLBACK.

JOIN und WHERE sind Bestandteile einer SELECT-Anweisung, aber keine eigenständigen Sprachfamilien.

CREATE TABLE: Grundmuster

```
CREATE TABLE Tabelle (  
    id            INTEGER      NOT NULL,  
    name          VARCHAR(100) NOT NULL,  
    betrag        DECIMAL(10,2),  
    status        VARCHAR(20)  DEFAULT 'offen',  
    CONSTRAINT t_pk PRIMARY KEY (id),  
    CONSTRAINT t_status_ck CHECK (status IN ('offen', 'erledigt'))  
);
```

Gängige Datentypen:

- INTEGER, NUMBER(p), DECIMAL(p,s) für Zahlen,
- CHAR(n) für feste Länge,
- VARCHAR(n)/VARCHAR2(n) für variable Länge,
- DATE, TIME, TIMESTAMP,
- CLOB, BLOB für große Texte/Binärdaten.

Die Unterlagen verwenden teilweise Oracle-Syntax (NUMBER, VARCHAR2). In einer Klausur sollte die im Beispiel verwendete Dialektform beibehalten werden.

Constraints

PRIMARY KEY

```
CONSTRAINT kunde_pk PRIMARY KEY (KNr)
```

Erzwingt UNIQUE und NOT NULL.

UNIQUE

```
CONSTRAINT professor_raum_uq UNIQUE (Raum)
```

Mehrere NULL-Werte sind je nach DBMS möglich; klausurrelevant ist vor allem die Eindeutigkeit nicht-NULLer Werte.

NOT NULL

Wird direkt an der Spalte notiert:

```
Name VARCHAR(50) NOT NULL
```

CHECK

```
CONSTRAINT transaktion_betrag_ck CHECK (Betrag > 0)
```

CHECK prüft pro Zeile. Subqueries und tabellenübergreifende Regeln sind in Standard-SQL-Checks meist nicht zulässig.

FOREIGN KEY

```
CONSTRAINT konto_kunde_fk  
  
FOREIGN KEY (KNr)  
  
REFERENCES Kunde(KNr)
```

Bei zusammengesetzten Schlüsseln müssen Anzahl, Reihenfolge und kompatible Datentypen übereinstimmen.

Referentielle Aktionen

Beim Löschen oder Ändern eines Elterntupels:

- `NO ACTION/RESTRICT`: Operation ablehnen, solange Kinder existieren.
- `ON DELETE CASCADE`: abhängige Kindtupel ebenfalls löschen.
- `ON DELETE SET NULL`: FK der Kinder auf NULL setzen; FK muss nullable sein.
- `ON DELETE SET DEFAULT`: Standardwert einsetzen; dieser muss referentiell gültig sein.
- `ON UPDATE ...`: analog, nicht in jedem DBMS gleich unterstützt.

Richtung merken: Die Aktion wird am FK der Kindtabelle definiert, ausgelöst wird sie durch UPDATE/DELETE in der Elterntabelle.

Wann CASCADE sinnvoll ist

Sinnvoll bei existenzabhängigen Daten, etwa:

- `Bestellung -> Bestellposition`,
- `Athlet -> Training` bei echter schwacher Entität,

- **Konto -> Transaktion**, wenn Transaktionen fachlich tatsächlich mitgelöscht werden dürfen.

Gefährlich bei historisch oder rechtlich aufbewahrungspflichtigen Daten. In echten Systemen würde man Transaktionen meist nicht kaskadierend löschen; in Klausuren gilt die vorgegebene Fachregel.

DDL-Beispiel: Kunde, Konto, Transaktion

Aus KLA1:

```
CREATE TABLE Konto (
    Nr                NUMBER,
    Eroeffnungsdatum DATE NOT NULL,
    KNr               NUMBER NOT NULL,
    CONSTRAINT konto_pk PRIMARY KEY (Nr),
    CONSTRAINT konto_kunde_fk
        FOREIGN KEY (KNr)
        REFERENCES Kunde(KNr)
        ON DELETE RESTRICT
);
```

Da ein Kunde mit vorhandenen Konten nicht gelöscht werden soll, bleibt die Standardaktion **RESTRICT/NO ACTION**; je nach Dialekt wird sie nicht ausdrücklich notiert.

Transaktion mit zwei Rollen zur selben Tabelle:

```
CREATE TABLE Transaktion (
    TNr              NUMBER PRIMARY KEY,
    Betrag           NUMBER NOT NULL,
    Zeitpunkt        DATE NOT NULL,
    Quelle           NUMBER NOT NULL,
    Ziel             NUMBER NOT NULL,
    CONSTRAINT trans_betrag_ck CHECK (Betrag > 0),
    CONSTRAINT trans_quelle_fk
        FOREIGN KEY (Quelle) REFERENCES Konto(Nr) ON DELETE CASCADE,
    CONSTRAINT trans_ziel_fk
        FOREIGN KEY (Ziel) REFERENCES Konto(Nr) ON DELETE CASCADE
);
```

Die Constraint-Namen erleichtern Fehlermeldungen, Wartung und gezieltes `DROP CONSTRAINT`.

Schemaevolution mit ALTER TABLE

```
ALTER TABLE Karte  
  
ADD CONSTRAINT karte_typ_ck  
  
CHECK (Typ IN ('Kundenkarte', 'EC-Karte', 'VISA', 'Mastercard'));
```

Weitere typische Formen:

```
ALTER TABLE Kunde ADD Email VARCHAR(255);  
  
ALTER TABLE Kunde DROP COLUMN Email;  
  
ALTER TABLE Kunde DROP CONSTRAINT kunde_email_uq;  
  
ALTER TABLE Kunde ADD CONSTRAINT kunde_email_uq UNIQUE (Email);
```

Syntaxdetails unterscheiden sich je DBMS; in der Klausur den Stil der Aufgabenstellung übernehmen.

Altklausur: Einkauf

Für die Tabelle `Einkauf` aus dem Supermarktmodell:

```
CREATE TABLE Einkauf (  
  
    BelegNr    NUMBER(10) NOT NULL,  
  
    Datum      DATE       NOT NULL,  
  
    Kassa      NUMBER(3)  NOT NULL,  
  
    KartenNr   NUMBER(16),  
  
    CONSTRAINT einkauf_pk PRIMARY KEY (BelegNr),  
  
    CONSTRAINT einkauf_karte_fk  
        FOREIGN KEY (KartenNr)  
        REFERENCES Karte(KartenNr)  
  
);
```

`KartenNr` bleibt nullable, weil NULL eine Barzahlung repräsentiert.

DML: INSERT, UPDATE, DELETE

INSERT

```
INSERT INTO Produkt (ProduktNr, Bezeichnung, Preis, Einheit)
VALUES (1005, 'Tee', 3.49, 'packung');
```

Spaltenliste immer bevorzugen. Ohne Spaltenliste müssen alle Spalten in Tabellendefinitionsreihenfolge angegeben werden.

INSERT ... SELECT

```
INSERT INTO ArchivKunde (KNr, Name)
SELECT KNr, Nachname
FROM Kunde
WHERE Status = 'inaktiv';
```

UPDATE

```
UPDATE Produkt
SET Preis = Preis * 1.03
WHERE Abteilung = 23;
```

Ohne `WHERE` werden alle Zeilen geändert.

DELETE

```
DELETE FROM Vorlesung
WHERE gelesenVon = 2125;
```

Auch hier: ohne `WHERE` werden alle Zeilen gelöscht, die Tabelle selbst bleibt bestehen. `DROP TABLE` entfernt dagegen das Schemaobjekt.

Selbstreferenzierende CASCADE-Kette

Bei:

```
FOREIGN KEY (Vorgesetzter)
REFERENCES Vorgesetzte(Angestellter)
ON DELETE CASCADE
```

kann das Löschen eines Vorgesetzten rekursiv alle direkt und indirekt abhängigen Untergebenen löschen. Prüfe die Kette grafisch als Baum. Zyklen oder Selbstreferenzen (Schulz -> Schulz) können DBMS-spezifische Besonderheiten auslösen; in Klausuraufgaben wird meist eine eindeutige Kaskadenfolge erwartet.

Selbsttest 10: Warum ist `ON DELETE SET NULL` für eine als `NOT NULL` definierte FK-Spalte unbrauchbar?

Lösung: Die referentielle Aktion würde einen Wert erzeugen, der die `NOT NULL`-Bedingung verletzt. Die Tabellendefinition ist semantisch widersprüchlich bzw. die Löschoperation würde scheitern.

Relationale Algebra

Grundidee und Abgeschlossenheit

Die relationale Algebra ist operational: Eine Anfrage wird als Folge von Operationen beschrieben. Jede Operation erhält Relationen und liefert wieder eine Relation. Dadurch können Ausdrücke beliebig verschachtelt werden.

Wichtige Operatoren:

Operator	Symbol	Zweck
Selektion	σ	Zeilen filtern
Projektion	π	Spalten auswählen, Duplikate im mathematischen Modell entfernen
Umbenennung	ρ oder β	Relation/Attribute umbenennen
kartesisches Produkt	$\times$	alle Tupelpaare
Join	$\bowtie$	passende Tupel verbinden
Vereinigung	$\cup$	Tupel aus beiden Relationen
Differenz	$-$	Tupel aus erster, nicht in zweiter Relation
Schnitt	$\cap$	gemeinsame Tupel
Gruppierung/Aggregation	γ	Gruppen und Aggregate
Sortierung	τ	in erweiterten Algebren sortieren

Selektion und Projektion

```
 $\sigma_{\text{Semester} < 5}(\text{Studenten})$ 
```

liefert alle Studenten unter dem 5. Semester.

```
 $\pi_{\text{Name}}(\sigma_{\text{Semester} < 5}(\text{Studenten}))$ 
```

liefert nur deren Namen.

Reihenfolge: Selektionen möglichst früh anwenden, um Zwischenergebnisse klein zu halten. Projektionen dürfen nur Attribute entfernen, die später nicht mehr für Join oder Ausgabe gebraucht werden.

Produkt und Join

Ein Theta-Join ist Produkt plus Bedingung:

```
 $R \bowtie_{R.a=S.b} S$ 
```

Der natürliche Join verbindet automatisch gleichnamige Attribute und führt diese im Ergebnis nur einmal.

Er ist bequem, aber gefährlich, wenn zufällig gleichnamige Attribute semantisch nicht zusammengehören.

Umbenennung und Selbstjoin

Für „Mitarbeiter mit Gutachten zu mindestens zwei verschiedenen Fahrzeugen“ wird dieselbe Relation zweimal benötigt:

```
πG1.mid(  
  σG1.mid = G2.mid ∧ G1.nr ≠ G2.nr(  
    ρG1(Gutachten) × ρG2(Gutachten)  
  )  
)
```

Die Ungleichheit der Fahrzeugnummern ist entscheidend; zwei Gutachten zum selben Fahrzeug zählen nicht.

Mengenoperationen und Kompatibilität

$R \cup S$, $R - S$ und $R \cap S$ setzen Vereinigungsverträglichkeit voraus:

- gleiche Anzahl Attribute,
- kompatible Domänen in entsprechender Position.

Attributnamen können vorher umbenannt werden.

„Keine“-Anfragen mit Differenz

Alle Vorlesungen ohne Voraussetzungen:

```
πVorlNr(Vorlesungen) - πVorlNr(πVorg(voraussetzen))
```

Je nach Schema kann statt `Vorg` die Seite verwendet werden, die tatsächlich „hat eine Voraussetzung“ bedeutet. Entscheidend ist die korrekte Rollensemantik.

„Alle Kombinationen minus vorhandene“

Altklausur: Für jeden Tester alle Modelle, die weder beendet noch begonnen wurden.

1. Alle möglichen Tester-Modell-Paare:

```
Alle := πSVNR(Tester) × πName(Modell)
```

2. Bereits begonnene Paare:

```
Begonnen := πSVNR, Name(beginnt_test)
```

3. Bereits beendete Paare:

```
Beendet :=  $\pi$ SVNR, Name(beendet_test)
```

4. Gesucht:

```
Alle - (Begonnen  $\cup$  Beendet)
```

Wenn jeder beendete Test zwingend zuvor begonnen wurde und der Beginn-Datensatz erhalten bleibt, reicht eventuell `Alle - Begonnen`. Die Aufgabenbeschreibung sollte entscheiden.

„Begonnen, aber nicht beendet“

Für Tester mit Nasengröße mindestens 3:

```
T :=  $\pi$ SVNR( $\sigma$ Nasengroesse >= 3(Tester))  
B :=  $\pi$ SVNR, Name(beginnt_test)  
E :=  $\pi$ SVNR, Name(beendet_test)  
(T  $\bowtie$  B) - (T  $\bowtie$  E)
```

Alternativ zuerst `B - E` und dann Join mit den gefilterten Testern.

Aggregation

Beispiel Anzahl Studenten pro Vorlesung:

```
yVorlNr; COUNT(MatNr) -> Anzahl(hoeren)
```

Die genaue Symbolsyntax variiert. Wichtig sind Gruppierungsattribute, Aggregat und Ergebnisname.

Division und Allquantifizierung

Die Division beantwortet „für alle“-Fragen. Beispiel: Welche Studenten hören **alle** Pflichtvorlesungen?

```
Hoeren(MatNr, VorlNr)  $\div$  Pflicht(VorlNr)
```

Ohne Divisionsoperator kann man mit Differenz formulieren:

Es darf keine Pflichtvorlesung geben, die der Student nicht hört.

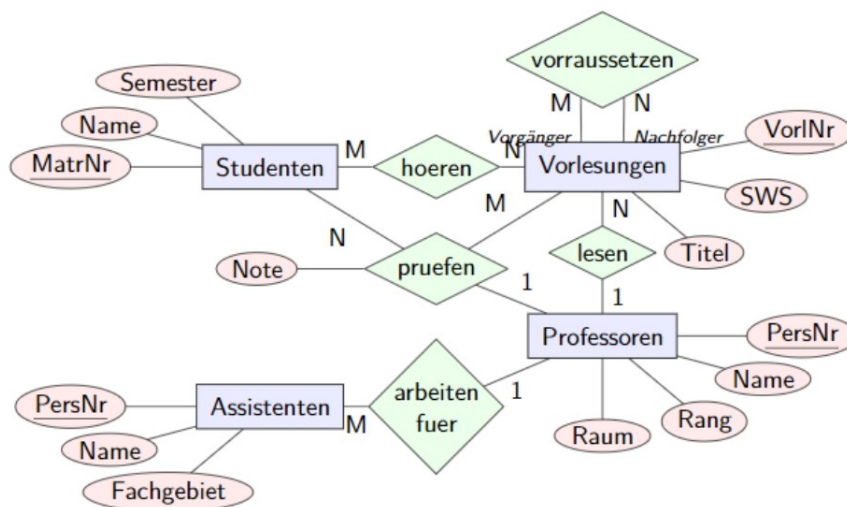
Konstruktiv:

```
AllePaare :=  $\pi$ MatNr(Studenten)  $\times$  Pflicht  
Fehlend := AllePaare - Hoeren  
StudentenMitFehlend :=  $\pi$ MatNr(Fehlend)  
Ergebnis :=  $\pi$ MatNr(Studenten) - StudentenMitFehlend
```

Universitätsbeispiele aus KLA1

Universitätsschema

In den nächsten beiden Aufgaben kommt wieder die Universitätsdatenbank aus der Vorlesung zum Einsatz.



Studenten		
MatrNr	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

Professoren			
PersNr	Name	Rang	Raum
2125	Sokrates	C4	226
2126	Russel	C4	232
2127	Kopernikus	C3	310
2133	Popper	C3	52
2134	Augustinus	C3	309
2136	Curie	C4	36
2137	Kant	C4	7

prüfen			
MatrNr	VorlNr	PersNr	Note
28106	5001	2126	1
25403	5041	2125	2
27550	4630	2137	2

voraussetzen	
Vorg.	Nachf.
5001	5041
5001	5043
5001	5049
5041	5216
5043	5052
5041	5052
5052	5259

hoeren	
MatrNr	VorlNr
26120	5001
27550	5001
27550	4052
28106	5041
28106	5001
28106	4052
28106	4630
29120	5001
29120	5041
29120	5049
29555	5022
29555	5001
25403	5022

Vorlesungen		
VorlNr	Titel	SWS
5001	Grundzüge	4
5041	Ethik	4
5043	Erkenntnistheorie	3
5049	Mäeutik	2
4052	Logik	4
5052	Wissenschaftstheorie	3
5216	Bioethik	2
5259	Der Wiener Kreis	2
5022	Glaube und Wissen	2
4630	Die 3 Kritiken	4

Assistenten			
PersNr	Name	Fachgebiet	Boss
3002	Platon	Ideenlehre	2125
3003	Aristoteles	Syllogistik	2125
3004	Wittgenstein	Sprachtheorie	2126
3005	Rhetikus	Planetenbewegung	2127
3006	Newton	Kepler Gesetze	2127
3007	Spinoza	Gott und Natur	2134

Universitätsschema und Datenbasis aus der Klausur vom 7. Oktober 2021.

Namen der Studenten, die Professor Popper aus Vorlesungen kennen

```

πStudenten.Name(
    Studenten
    ⋈ Studenten.MatrNr = hoeren.MatrNr hoeren
    ⋈ hoeren.VorlNr = Vorlesungen.VorlNr Vorlesungen
    ⋈ Vorlesungen.GelesenVon = Professoren.PersNr
    σProfessoren.Name = 'Popper'(Professoren)
)
    
```

Eine saubere Schreibweise kann Selektion und Joins anders klammern; semantisch müssen die Pfade

stimmen.

Räume der C3-Professoren

```
 $\pi_{\text{Raum}}(\sigma_{\text{Rang} = 'C3'}(\text{Professoren}))$ 
```

Vorlesungen ohne Voraussetzungen

```
 $\pi_{\text{Vor\text{Nr}}}(\text{Vorlesungen}) - \rho_{\text{Vor\text{Nr}}}(\pi_{\text{Nachf}}(\text{voraussetzen}))$ 
```

Hier sind die Vorlesungen mit einem Eintrag als Nachfolger diejenigen, die mindestens eine Voraussetzung haben.

Algebra-Checkliste

1. Welche Relation enthält das Ausgabeattribut?
2. Welche Joinpfade verbinden die benötigten Relationen?
3. Welche Bedingungen sind Zeilenfilter?
4. Brauche ich Umbenennung/Selbstjoin?
5. Bedeutet „keine“ eine Differenz oder `NOT EXISTS`-Logik?
6. Bedeutet „mindestens zwei verschiedene“ einen Selbstjoin mit Ungleichheit?
7. Bedeutet „alle“ Division oder doppelte Differenz?
8. Ist das Endschema exakt wie gefordert?

Selbsttest 11: Formuliere die IDs der Mitarbeiter, deren Stufe kleiner als das Level eines von ihnen erstellten Gutachtens ist.

Lösung:

```
 $\pi_{\text{mid}}(\sigma_{\text{stufe} < \text{lvl}}(\text{Gutachten} \bowtie \text{Gutachten.mid} = \text{Mitarbeiter.mid} \text{ Mitarbeiter} \\ \bowtie \text{Gutachten.name} = \text{Zertifikat.name} \text{ Zertifikat}))$ 
```

SQL-Abfragen systematisch lösen

Der SFW-Kern und logische Auswertungsreihenfolge

Grundform:

```
SELECT [DISTINCT] Ausdrucksliste  
  
FROM Tabellen und Joins  
  
WHERE Zeilenbedingung  
  
GROUP BY Gruppierungsattribute  
  
HAVING Gruppenbedingung  
  
ORDER BY Sortierausdrücke;
```

Logische Reihenfolge:

1. FROM/JOIN
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. DISTINCT
7. ORDER BY

Deshalb kann ein Alias aus `SELECT` häufig nicht in `WHERE` verwendet werden: Er entsteht logisch erst später. In `ORDER BY` ist er meist erlaubt.

Einfache Projektion, Ausdruck und Alias

Altklausur, Gesamtbestand für Abteilung 23:

```
SELECT ProduktNr,  
       Bezeichnung,  
       Abteilung,  
       Lagerbestand + Regalbestand AS Gesamtbestand  
  
FROM Produkt  
  
WHERE Abteilung = 23;
```

`SELECT` kann Attribute, Konstanten, arithmetische Ausdrücke und Funktionen enthalten.

DISTINCT

```
SELECT DISTINCT KartenNr
FROM Einkauf
WHERE Datum = DATE '2019-01-05'
AND Kassa = 3
AND KartenNr IS NOT NULL
ORDER BY KartenNr DESC;
```

`DISTINCT` gilt auf die gesamte ausgewählte Zeilenkombination, nicht nur auf die erste Spalte.

NULL und dreiwertige Logik

NULL bedeutet je nach Kontext „unbekannt“, „nicht vorhanden“ oder „nicht anwendbar“. Es ist kein normaler Wert.

Falsch:

```
WHERE KartenNr = NULL
WHERE KartenNr <> NULL
```

Richtig:

```
WHERE KartenNr IS NULL
WHERE KartenNr IS NOT NULL
```

Vergleiche mit NULL ergeben `UNKNOWN`. `WHERE` behält nur Zeilen mit Ergebnis `TRUE`; `FALSE` und `UNKNOWN` werden verworfen.

COUNT und NULL

- `COUNT(*)` zählt Zeilen.
- `COUNT(Spalte)` zählt nur nicht-NULL-Werte.
- `COUNT(DISTINCT Spalte)` zählt verschiedene nicht-NULL-Werte.

Bar bezahlte Einkäufe 2019:

```
SELECT COUNT(*) AS Anzahl_Einkaeufe_Bar_2019
FROM Einkauf
WHERE KartenNr IS NULL
AND Datum >= DATE '2019-01-01'
AND Datum < DATE '2020-01-01';
```

Ein halboffenes Datumsintervall ist robuster als Funktionen auf der Datumsspalte und funktioniert auch bei Zeitanteilen.

Joins

Expliziter INNER JOIN

```
SELECT p.ProduktNr, p.Bezeichnung, ep.Anzahl
FROM Produkt p
JOIN Einkaufsposition ep
ON ep.ProduktNr = p.ProduktNr;
```

Explizite Join-Syntax trennt Verbindungsbedingungen von fachlichen Filtern und ist der Komma-Syntax vorzuziehen.

LEFT OUTER JOIN

Alle Vorlesungen einschließlich solcher ohne Hörer:

```
SELECT v.VorlNr,
       v.Titel,
       COUNT(h.MatrNr) AS AnzahlStudierende
FROM Vorlesungen v
LEFT JOIN hoeren h
ON h.VorlNr = v.VorlNr
GROUP BY v.VorlNr, v.Titel;
```

COUNT(h.MatrNr) liefert für eine Vorlesung ohne Treffer 0; COUNT(*) würde wegen der erhaltenen linken Zeile 1 liefern und wäre falsch.

Bedingungen bei OUTER JOIN

Ein Filter auf die rechte Tabelle in WHERE kann den Outer Join unbeabsichtigt in einen Inner Join verwandeln. Bedingungen, die nur die Join-Partnerauswahl betreffen, gehören oft in ON.

Aggregatfunktionen und GROUP BY

Aggregatfunktionen: COUNT, SUM, AVG, MIN, MAX.

Regel: Jede nicht aggregierte Spalte im SELECT muss im GROUP BY stehen, sofern das DBMS keine funktionalen Abhängigkeiten speziell berücksichtigt.

Produkt mindestens in zwei Einkäufen

Altklausur: Produkte, die am 5. Januar 2019 an Kassa 3 bei mindestens zwei Einkäufen vorkamen:

```
SELECT p.ProduktNr,
       p.Bezeichnung,
       COUNT(DISTINCT e.BelegNr) AS AnzahlEinkaeufe
FROM Produkt p
JOIN Einkaufsposition ep
  ON ep.ProduktNr = p.ProduktNr
JOIN Einkauf e
  ON e.BelegNr = ep.BelegNr
WHERE e.Datum = DATE '2019-01-05'
      AND e.Kassa = 3
GROUP BY p.ProduktNr, p.Bezeichnung
HAVING COUNT(DISTINCT e.BelegNr) >= 2;
```

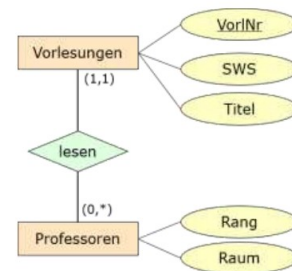
Warum `DISTINCT e.BelegNr`? Das Ergebnis soll Einkäufe zählen, nicht Positionen. Bei sauberem PK (`BelegNr, ProduktNr`) gibt es pro Produkt und Einkauf zwar nur eine Position, doch `DISTINCT` macht die Absicht explizit.

WHERE oder HAVING?

- `WHERE` filtert **Zeilen vor** der Gruppierung.
- `HAVING` filtert **Gruppen nach** der Aggregation.

Beispiel: Nur C4-Professoren betrachten (`WHERE Rang='C4'`), dann Gruppen behalten, deren durchschnittliche SWS mindestens 3 sind (`HAVING AVG(SWS)>=3`).

Aggregate und Gruppierung



Beispiel:

Geben Sie zu jedem/r C4-ProfessorIn, der/die vor allem lange Vorlesungen (Durchschnitt größer gleich 3) hält, die Summe der von ihm/ihr gehaltenen Vorlesungsstunden an?

```
SELECT gelesenVon, Name, SUM(SWS)
FROM Vorlesung INNER JOIN Professor ON gelesenVon = PersNr
WHERE Rang = 'C4'
GROUP BY gelesenVon, Name
HAVING AVG(SWS) >= 3;
```

Unterabfragen

Skalare Unterabfrage

Mitarbeiter mit Gehalt über dem Gesamtdurchschnitt:

```
SELECT employee_id, first_name, last_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Die Unterabfrage muss höchstens einen Wert liefern.

IN

```
SELECT Name
FROM Studenten
WHERE MatrNr IN (
    SELECT MatrNr
    FROM hoeren)
```

```
WHERE VorlNr = 5001
```

```
);
```

EXISTS

```
SELECT v.VorlNr, v.Titel
FROM Vorlesungen v
WHERE EXISTS (
    SELECT 1
    FROM hoeren h
    WHERE h.VorlNr = v.VorlNr
);
```

EXISTS prüft nur, ob mindestens eine Zeile existiert. Die ausgewählte Konstante `1` hat keine fachliche Bedeutung.

NOT EXISTS

Vorlesungen ohne Voraussetzung:

```
SELECT v.VorlNr, v.Titel
FROM Vorlesungen v
WHERE NOT EXISTS (
    SELECT 1
    FROM voraussetzen x
    WHERE x.Nachf = v.VorlNr
);
```

`NOT EXISTS` ist bei NULL-Problemen oft sicherer als `NOT IN`.

Korrelierte Unterabfragen

Eine korrelierte Unterabfrage verweist auf eine Spalte der äußeren Abfrage. Konzeptuell wird sie für jede äußere Zeile ausgewertet, auch wenn der Optimierer intern umformt.

Mitarbeiter, die mehr als der Durchschnitt ihrer eigenen Abteilung verdienen:

```
SELECT e.first_name, e.last_name, e.salary, e.department_id
FROM employees e
```

```

WHERE e.salary > (

  SELECT AVG(esub.salary)

  FROM employees esub

  WHERE esub.department_id = e.department_id

);

```

Der Alias `e` aus der äußeren Abfrage macht die Unterabfrage korreliert.

ANY, ALL und Allquantifizierung

- `x > ANY (Menge)`: größer als mindestens ein Element.
- `x > ALL (Menge)`: größer als jedes Element.
- `x = ANY (...)` entspricht inhaltlich häufig `IN`.

„Student hört alle Pflichtvorlesungen“ wird robust mit doppeltem `NOT EXISTS` formuliert:

```

SELECT s.MatrNr, s.Name

FROM Studenten s

WHERE NOT EXISTS (

  SELECT 1

  FROM Pflichtvorlesung p

  WHERE NOT EXISTS (

    SELECT 1

    FROM hoeren h

    WHERE h.MatrNr = s.MatrNr

    AND h.VorlNr = p.VorlNr

  )

);

```

Lesart: Es gibt keine Pflichtvorlesung, für die kein passender Hör-Eintrag existiert.

Mengenoperationen in SQL

```

SELECT PersNr FROM Professoren

UNION

SELECT PersNr FROM Assistenten;

```

`UNION` entfernt Duplikate. `UNION ALL` behält sie und ist meist effizienter.

Weitere Operatoren: `INTERSECT`, `EXCEPT` bzw. Oracle `MINUS`.

KLA1-Aufgabe „alle Personalnummern von Professoren und Assistenten“:

```
SELECT PersNr FROM Professoren

UNION

SELECT PersNr FROM Assistenten;
```

CASE und bedingte Ausgaben

```
SELECT BelegNr,

       CASE

         WHEN KartenNr IS NULL THEN 'Bar'

         ELSE 'Karte'

       END AS Zahlungsart

FROM Einkauf;
```

`CASE` ist ein Ausdruck und kann in `SELECT`, `ORDER BY` oder Aggregationen verwendet werden.

Zeichenketten und Muster

```
WHERE Nachname LIKE 'M%'

WHERE Code LIKE 'A_7'
```

- `%` beliebig viele Zeichen,
- `_` genau ein Zeichen.

Mit `ESCAPE` kann ein Platzhalter als normales Zeichen gesucht werden.

Datumsbedingungen

Vermeide, wenn möglich, Funktionen auf der gefilterten Spalte:

```
-- robust und indexfreundlich

WHERE Datum >= DATE '2019-01-01'

AND Datum < DATE '2020-01-01'
```

statt:

```
WHERE EXTRACT(YEAR FROM Datum) = 2019
```

Die zweite Variante kann einen normalen Index auf `Datum` schlechter nutzbar machen.

SQL-Fehlermuster

1. `= NULL` statt `IS NULL`.
2. `WHERE COUNT(*) > 2` statt `HAVING COUNT(*) > 2`.
3. Spalten im `SELECT`, die weder gruppiert noch aggregiert sind.
4. fehlende Joinbedingung -> kartesisches Produkt.
5. `COUNT(*)` bei `LEFT JOIN`, obwohl Nulltreffer als 0 gezählt werden sollen.
6. `NOT IN` mit möglichem `NULL` in der Unterabfrage.
7. `ORDER BY` mit `GROUP BY` verwechseln.
8. bei „verschiedene“ kein `DISTINCT` oder keine Ungleichheitsbedingung.
9. bei „mindestens zwei Einkäufe“ Positionen statt Belege zählen.
10. Filter der rechten Outer-Join-Tabelle in `WHERE` platzieren.

Selbsttest 12: Gib die Namen der Studenten aus, die weniger als fünf Semester studieren.

Lösung:

```
SELECT Name
FROM Studenten
WHERE Semester < 5;
```

Selbsttest 13: Gib die Anzahl der Studenten pro Vorlesung aus, einschließlich Vorlesungen ohne Studenten.

Lösung:

```
SELECT v.VorlNr, v.Titel, COUNT(h.MatrNr) AS Anzahl
FROM Vorlesungen v
LEFT JOIN hoeren h ON h.VorlNr = v.VorlNr
GROUP BY v.VorlNr, v.Titel;
```

Selbsttest 14: Welche Abteilungen haben ein Durchschnittsgehalt über 10.000? Gib Abteilungsname, Mitarbeiterzahl und Durchschnitt aus.

Lösung:

```
SELECT d.department_name,
       COUNT(e.employee_id) AS anzahl,
       AVG(e.salary) AS durchschnitt
FROM departments d
JOIN employees e ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name
HAVING AVG(e.salary) > 10000;
```

Sichten

Was ist eine Sicht?

Eine virtuelle Sicht ist eine benannte Abfrage, die wie eine Tabelle verwendet werden kann:

```
CREATE VIEW C4_Professoren AS

SELECT PersNr, Name, Raum

FROM Professoren

WHERE Rang = 'C4';
```

Die Daten einer normalen Sicht werden nicht dauerhaft als eigener Tabelleninhalt gespeichert; die Abfrage wird bei Verwendung logisch auf die Basistabellen zurückgeführt.

Einsatzgebiete:

- komplexe Abfragen vereinfachen,
- benutzerspezifische Datenmodelle anbieten,
- logische Datenunabhängigkeit erhöhen,
- Spalten oder Zeilen aus Sicherheitsgründen ausblenden,
- ein stabiles Interface über veränderlichen Basistabellen schaffen.

Sichtarten

- **Selektionssicht:** filtert Zeilen.
- **Projektionssicht:** wählt Spalten.
- **Verbundssicht:** verbindet mehrere Tabellen.
- **Aggregationssicht:** enthält Gruppierungen/Aggregate.

Sichtarten und Einschränkungen bzgl. Änderbarkeit

- Vier verschiedene Sichtarten
 - Projektion ⇒ **Projektionssicht**
 - Selektion ⇒ **Selektionssicht**
 - Verbund ⇒ **Verbundssicht**
 - Aggregation ⇒ **Aggregationssicht**
- Faustregel
 - Auf einer Sicht sind nur solche DML-Operationen zulässig, zu denen eine **eindeutige Transformation auf die Basisrelationen** existiert.

Änderbarkeit

Faustregel:

Eine DML-Operation auf einer Sicht ist nur möglich, wenn sie eindeutig auf Basistabellenzeilen abgebildet werden kann.

Meist gut änderbar:

- einfache Sicht auf eine Basistabelle,
- keine Aggregate,
- kein **DISTINCT**, **GROUP BY**, Mengenoperator,
- alle notwendigen NOT-NULL-Spalten ohne Default sind verfügbar.

Problematisch oder nicht änderbar:

- Aggregationssichten,
- viele Verbundssichten,
- Sichten mit berechneten Spalten,
- **UNION**, **DISTINCT**, Gruppierung.

WITH CHECK OPTION verhindert, dass über die Sicht Zeilen erzeugt oder geändert werden, die anschließend aus der Sicht herausfallen:

```
CREATE VIEW C4_Professoren AS
```

```
SELECT PersNr, Name, Rang, Raum  
  
FROM Professoren  
  
WHERE Rang = 'C4'  
  
WITH CHECK OPTION;
```

Materialisierte Sicht

Eine materialisierte Sicht speichert das Ergebnis physisch und wird nach einer Strategie aktualisiert. Sie kann komplexe Abfragen beschleunigen, ist aber potenziell zeitlich veraltet und benötigt Speicher sowie Refresh-Logik.

Selbsttest 15: Warum ist eine Sicht mit `GROUP BY Abteilung` und `AVG(Gehalt)` normalerweise nicht direkt updatefähig?

Lösung: Eine sichtbare Aggregatzelle steht für mehrere Basistupel. Eine Änderung des Durchschnittswerts lässt sich nicht eindeutig auf einzelne Mitarbeitergehälter zurückübersetzen.

Physische Datenorganisation und Indizes

Zugriff ohne und mit Index

Ohne passenden Index muss das DBMS möglicherweise viele oder alle Datenseiten lesen. Ein Index ist eine zusätzliche Datenstruktur mit Suchschlüsseln und Zeigern auf Datensätze oder Datenblöcke.

Typische Zugriffe:

- Punktanfrage: `WHERE KNr = 33,`
- Bereichsanfrage: `WHERE Umsatz > 100000,`
- Join über FK/PK,
- Sortierung und Min/Max.

Indizes beschleunigen Lesen, verursachen aber Kosten:

- Speicherplatz,
- Pflege bei `INSERT`, `UPDATE`, `DELETE`,
- zusätzliche Planungskomplexität.

B+-Baum-Idee

Ein mehrstufiger balancierter Suchbaum reduziert die Anzahl gelesener Blöcke. Innere Knoten führen zu Blattseiten; Blattseiten enthalten Schlüssel und Zeiger oder bei Clusterung die Daten selbst. Bereichsanfragen profitieren von verketteten Blattseiten.

Primär- und Sekundärindex

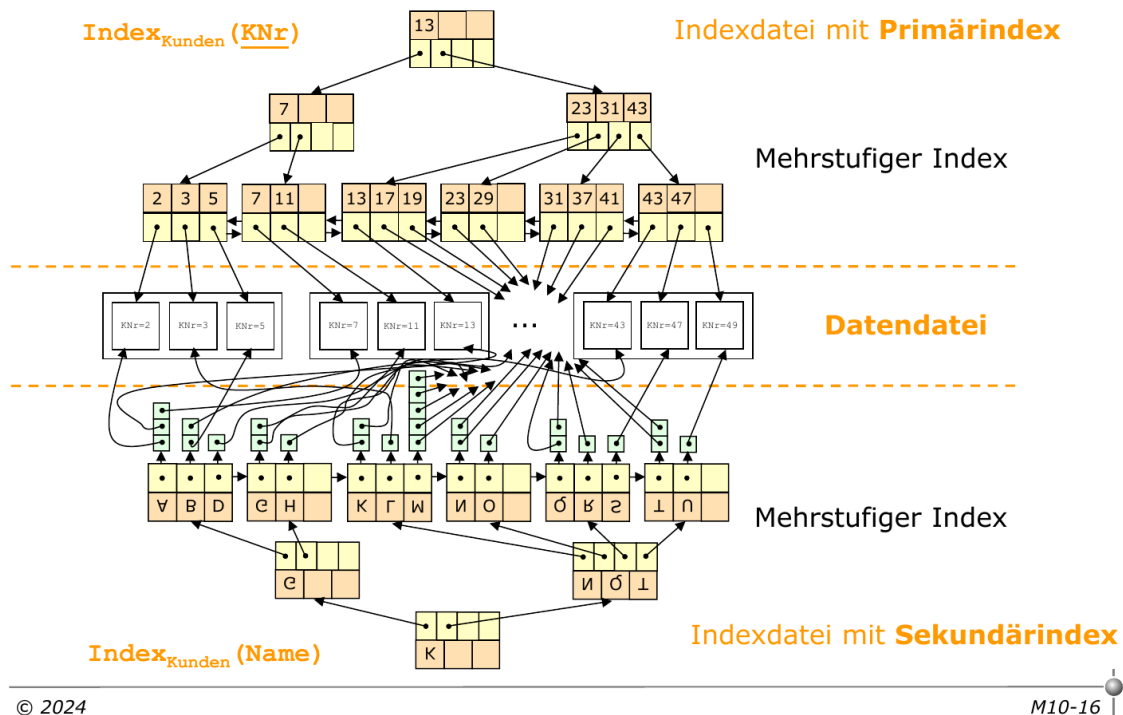
Im Kursgebrauch:

- **Primärindex:** häufig automatisch über Primärschlüssel/UNIQUE, eindeutig; oft eng mit physischer Ordnung verbunden.
- **Sekundärindex:** zusätzlicher Zugriffspfad auf andere Spalten; muss nicht eindeutig sein.

Nur ein Index kann die physische Reihenfolge einer Tabelle direkt bestimmen; mehrere Sekundärindizes sind möglich.

Indexstrukturen

Kombination Primär- und Sekundärindex



© 2024

M10-16

Kombination von Primär- und Sekundärindex. Quelle: M10_Physische_Datenorganisation, Folie 16.

Gute Indexkandidaten

- Primärschlüssel und UNIQUE-Spalten,
- häufig verwendete Fremdschlüssel, besonders für Joins,
- selektive WHERE-Bedingungen,
- häufige Bereichssuchen,
- Spalten für ORDER BY, MIN, MAX, sofern der Zugriffspfad passt.

Schlechte Kandidaten:

- sehr kleine Tabellen,
- Spalten mit sehr wenigen verschiedenen Werten, wenn große Ergebnismengen entstehen,
- Spalten mit sehr häufigen Änderungen,
- Bedingungen, die durch Funktionen den Indexschlüssel verändern, sofern kein Funktionsindex existiert.

Selektivität

Selektivität beschreibt grob, wie klein der Trefferanteil ist. Ein Index lohnt sich besonders, wenn nur ein kleiner Teil der Tabelle gesucht wird. `Geschlecht = 'X'` in einer gleich verteilten riesigen Tabelle ist oft wenig selektiv; `Kundennummer = 4711` ist sehr selektiv.

Mehrspaltige Indizes und Reihenfolge

Index (A,B) unterstützt typischerweise gut:

- WHERE A = ...,
- WHERE A = ... AND B = ...,
- oft ORDER BY A,B.

Er unterstützt WHERE B = ... nicht automatisch ebenso gut, weil die führende Spalte A fehlt. Die Spaltenreihenfolge folgt den wichtigsten Zugriffswegen, nicht der optischen Reihenfolge im Schema.

CREATE INDEX

SQL-Standard und DBMS unterscheiden sich; eine typische Form ist:

```
CREATE INDEX einkauf_datum_kassa_ix  
ON Einkauf(Datum, Kassa);
```

Eindeutig:

```
CREATE UNIQUE INDEX kunde_email_uix  
ON Kunde(Email);
```

In der Praxis ist ein UNIQUE-Constraint fachlich aussagekräftiger als nur ein eindeutiger Index, wenn echte Integrität gemeint ist.

Selbsttest 16: Eine Abfrage sucht fast immer WHERE Datum >= ? AND Datum < ? AND Kassa = ?. Welcher Index ist plausibel?

Lösung: Je nach Verteilung etwa (Kassa, Datum) oder (Datum, Kassa). Wenn Kassa sehr selektiv ist und danach ein Datumsbereich folgt, ist (Kassa, Datum) oft günstig. Die endgültige Entscheidung benötigt Statistiken und Ausführungspläne.

Kompakte Prüfungssimulation

Die folgenden Aufgaben kombinieren typische Muster aus der Klausurvorbereitung und den beiden Altklausuren. Bearbeite sie zuerst ohne Lösung.

Aufgabe A - Konzeptuelle Modellierung

Eine Bibliothek verwaltet Bücher und Zeitschriften. Ein Buch ist durch ISBN identifiziert und kann mehrere Autoren besitzen. Die Bibliothek besitzt mehrere Exemplare eines Buches; eine Exemplarnummer ist nur innerhalb einer ISBN eindeutig. Zeitschriften erscheinen in Heften und werden jahrgangsweise gebunden. Bücher und Artikel sind einem oder mehreren Fachgebieten zugeordnet. Nur Buchexemplare können von Lesern ausgeliehen werden; zu jeder Ausleihe sind Beginn und Rückgabedatum zu speichern.

Arbeitsauftrag: Entitätstypen, Schlüssel, Beziehungen und Min-Max-Kardinalitäten in Chen-Notation angeben. Schwache Entitäten kenntlich machen.

Lösungsskizze A

Entitäten:

- `Buch(ISBN, Titel, ...)`
- `Buchexemplar(ExemplarNr, Zustand)` schwach unter Buch
- `Autor(AutorNr, Name)`
- `Zeitschrift(ZNr, Titel)`
- `Heft(HeftNr, Jahr)` schwach unter Zeitschrift, falls HeftNr nur dort eindeutig
- `Artikel(ArtikelNr, Titel)` oder schwach unter Heft, je nach Nummerierung
- `Fachgebiet(FachNr, Bezeichnung)`
- `Leser(LeserNr, Name)`

Beziehungen:

- Buch M:N `verfasst_von` Autor
- Buch 1:N identifiziert Buchexemplar
- Zeitschrift 1:N identifiziert Heft
- Heft 1:N `enthält` Artikel
- Buch M:N `gehört_zu` Fachgebiet
- Artikel M:N `gehört_zu` Fachgebiet
- Leser M:N `leiht` Buchexemplar mit `Beginn`, `Rueckgabe`; falls nur eine gleichzeitige Ausleihe je Exemplar erlaubt, zusätzliche zeitabhängige Regel dokumentieren.

Aufgabe B - Relationale Abbildung

Überführe die Lösung aus A in Relationen. Verwende das Partitionierungsprinzip nicht, da keine Vererbung vorkommt.

Lösungsskizze B

```
Buch(ISBN PK, Titel)

Buchexemplar(ISBN PFK, ExemplarNr PK, Zustand, PK(ISBN,ExemplarNr))

Autor(AutorNr PK, Name)

BuchAutor(ISBN PFK, AutorNr PFK, PK(ISBN,AutorNr))

Zeitschrift(ZNr PK, Titel)

Heft(ZNr PFK, HeftNr PK, Jahr, PK(ZNr,HeftNr))

Artikel(ArtikelNr PK, ZNr FK, HeftNr FK, Titel)

Fachgebiet(FachNr PK, Bezeichnung)

BuchFachgebiet(ISBN PFK, FachNr PFK, PK(ISBN,FachNr))

ArtikelFachgebiet(ArtikelNr PFK, FachNr PFK, PK(ArtikelNr,FachNr))

Leser(LeserNr PK, Name)

Ausleihe(LeserNr PFK, ISBN PFK, ExemplarNr PFK, Beginn PK,
         Rueckgabe, PK(LeserNr,ISBN,ExemplarNr,Beginn))
```

Der Schlüssel der Ausleihe ist eine Annahme. Eine künstliche `AusleiheNr` wäre ebenfalls vertretbar, sofern künstliche Schlüssel erlaubt sind.

Aufgabe C - Funktionale Abhängigkeiten

Gegeben:

```
R(PNr, PName, AbtNr, AbtName, ProjektNr, ProjektName, Stunden)
```

Fachregeln:

```
PNr -> PName, AbtNr
AbtNr -> AbtName
ProjektNr -> ProjektName
(PNr, ProjektNr) -> Stunden
```

1. Bestimme einen Kandidatenschlüssel.
2. Zerlege in 3NF.

Lösung C

Kandidatenschlüssel: `(PNr,ProjektNr)`. Hülle enthält über `PNr` Person und Abteilung, über `AbtNr` Abteilungsname, über `ProjektNr` Projektname und direkt Stunden.

```

Person(PNr PK, PName, AbtNr FK)

Abteilung(AbtNr PK, AbtName)

Projekt(ProjektNr PK, ProjektName)

PersonProjekt(PNr PFK, ProjektNr PFK, Stunden,
              PK(PNr,ProjektNr))

```

Aufgabe D - DDL

Erzeuge `PersonProjekt` mit `Stunden > 0` und kaskadierendem Löschen, wenn eine Person gelöscht wird. Projekte sollen bei vorhandenen Zuordnungen nicht löscher sein.

Lösung D

```

CREATE TABLE PersonProjekt (

  PNr          INTEGER NOT NULL,

  ProjektNr    INTEGER NOT NULL,

  Stunden      DECIMAL(6,2) NOT NULL,

  CONSTRAINT pp_pk PRIMARY KEY (PNr, ProjektNr),

  CONSTRAINT pp_stunden_ck CHECK (Stunden > 0),

  CONSTRAINT pp_person_fk

    FOREIGN KEY (PNr) REFERENCES Person(PNr)

    ON DELETE CASCADE,

  CONSTRAINT pp_projekt_fk

    FOREIGN KEY (ProjektNr) REFERENCES Projekt(ProjektNr)

);

```

Aufgabe E - Relationale Algebra

Schema:

```

Produkt(ProduktNr, Bezeichnung, Abteilung)

Einkauf(BelegNr, Datum, Kassa)

Position(BelegNr, ProduktNr, Anzahl)

```

Formuliere Produktnummern der Produkte, die in mindestens zwei verschiedenen Einkäufen vorkamen.

Lösung E

```
πP1.ProduktNr(  
    σP1.ProduktNr=P2.ProduktNr ∧ P1.BelegNr≠P2.BelegNr(  
        ρP1(Position) × ρP2(Position)  
    )  
)
```

Aufgabe F - SQL

1. Gesamtmenge je Produkt: Summe aller `Anzahl` aus Positionen.
2. Produkte ohne Position.
3. Abteilungen mit mehr als fünf verschiedenen Produkten in Einkäufen.

Lösung F

```
-- 1  
  
SELECT p.ProduktNr, p.Bezeichnung,  
       COALESCE(SUM(ep.Anzahl), 0) AS Gesamtmenge  
  
FROM Produkt p  
  
LEFT JOIN Einkaufsposition ep ON ep.ProduktNr = p.ProduktNr  
  
GROUP BY p.ProduktNr, p.Bezeichnung;  
  
  
-- 2  
  
SELECT p.ProduktNr, p.Bezeichnung  
  
FROM Produkt p  
  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM Einkaufsposition ep  
    WHERE ep.ProduktNr = p.ProduktNr  
);  
  
  
-- 3  
  
SELECT p.Abteilung,  
       COUNT(DISTINCT p.ProduktNr) AS Produktanzahl
```

```
FROM Produkt p

JOIN Einkaufsposition ep ON ep.ProduktNr = p.ProduktNr

GROUP BY p.Abteilung

HAVING COUNT(DISTINCT p.ProduktNr) > 5;
```

Aufgabe G - Wahr oder falsch

1. Ein Fremdschlüssel muss immer **NOT NULL** sein.
2. Ein Schlüssel darf kein überflüssiges Attribut enthalten.
3. Eine Relation mit einattributigem Schlüssel ist automatisch in 2NF, wenn sie in 1NF ist.
4. Eine M:N-Beziehung kann ohne separate Relation durch einen einzelnen FK umgesetzt werden.
5. **COUNT(Spalte)** zählt NULL-Werte.
6. **HAVING** wird logisch vor **WHERE** ausgewertet.
7. Ein Untertyp besitzt im Partitionierungsmodell denselben Schlüsselwert wie sein Obertyp.
8. Ein sekundärer Index muss eindeutig sein.

Lösung G

1. falsch - optionale Beziehung erlaubt NULL.
2. richtig - Kandidatenschlüssel sind minimal.
3. richtig.
4. falsch.
5. falsch.
6. falsch - WHERE vor Gruppierung, HAVING danach.
7. richtig - PK des Untertyps ist zugleich FK.
8. falsch.

Klausurstrategie nach Aufgabentyp

ER-Modellierung

- Text zweimal lesen: zuerst Konzepte, dann Kardinalitätswörter.
- Schlüssel sofort unterstreichen.
- Erst Entitäten und Beziehungen zeichnen, dann Attribute ergänzen.
- Für jede Kante zwei Sätze formulieren.
- Spezialfälle sichtbar notieren.
- Annahmen in 2-4 klaren Sätzen dokumentieren.

Punkte retten: Ein unvollständiges, aber lesbares Modell mit korrekten Schlüsseln und Beziehungen ist besser als ein überladenes Diagramm ohne eindeutige Semantik.

Relationale Abbildung

Arbeite in dieser Reihenfolge:

1. starke Entitäten,
2. schwache Entitäten,
3. Vererbung,
4. 1:N,
5. M:N/n-stellig,
6. 1:1 und NULL-Vermeidung,
7. Attribute/Datentypen,
8. PK/FK/PFK markieren.

Danach kontrolliere jeden ER-Knoten und jede Beziehung: Wo ist diese Information im Schema geblieben?

Normalisierung

- Fachliche FDs ausdrücklich hinschreiben.
- Kandidatenschlüssel über Hülle bestimmen.
- Teilabhängigkeiten vor transitiven Abhängigkeiten suchen.
- Jede Zerlegung mit PK/FK versehen.
- Nicht nur Spalten verteilen, sondern fachliche Entitäten erkennen.

DDL und Integrität

- Tabellen in Eltern-vor-Kind-Reihenfolge definieren.
- Datentyp, NULL-Regel, Schlüssel und CHECK pro Spalte prüfen.
- Bei FK immer Zielrelation und Zielattribut nennen.
- CASCADE nur wenn gefordert.
- Constraint-Namen kurz und eindeutig halten.

Relationale Algebra

- Zielattribute ganz am Ende projizieren.
- Joinpfad markieren.
- „keine“ -> Differenz.
- „mindestens zwei verschiedene“ -> Selbstprodukt/Selbstjoin plus $\neq$.
- „alle“ -> Division oder doppelte Differenz.

SQL

Schreibe zunächst ein Gerüst:

```
SELECT ...  
  
FROM ...  
  
JOIN ... ON ...  
  
WHERE ...  
  
GROUP BY ...  
  
HAVING ...  
  
ORDER BY ...;
```

Fülle es in logischer Reihenfolge `FROM -> WHERE -> GROUP -> HAVING -> SELECT`. Prüfe danach NULL, Duplikate und Zählobjekt.

SQL-Merkblatt erweitert

SELECT

```
SELECT [DISTINCT] spalten_und_ausdruecke

FROM relationen

[JOIN relation ON bedingung]

[WHERE zeilenbedingung]

[GROUP BY gruppierungsattribute]

[HAVING gruppenbedingung]

[ORDER BY sortierausdruecke [ASC|DESC]];
```

Vergleich und Logik

```
= <> < <= > >=

AND OR NOT

BETWEEN ... AND ...

IN (...)

LIKE '%..._-'

IS NULL / IS NOT NULL

EXISTS / NOT EXISTS

ANY / ALL
```

Aggregate

```
COUNT(*)

COUNT(spalte)

COUNT(DISTINCT spalte)

SUM(spalte)

AVG(spalte)

MIN(spalte)
```

```
MAX(spalte)
```

DDL

```
CREATE TABLE ...;  
  
ALTER TABLE ... ADD ...;  
  
ALTER TABLE ... ADD CONSTRAINT ...;  
  
ALTER TABLE ... DROP CONSTRAINT ...;  
  
DROP TABLE ...;
```

DML

```
INSERT INTO t (a,b) VALUES (...,...);  
  
UPDATE t SET a=... WHERE ...;  
  
DELETE FROM t WHERE ...;
```

Schlüssel und Constraints

```
PRIMARY KEY (a,b)  
  
UNIQUE (a,b)  
  
FOREIGN KEY (a,b) REFERENCES eltern(x,y)  
  
CHECK (...)  
  
NOT NULL  
  
DEFAULT ...  
  
ON DELETE CASCADE | SET NULL | SET DEFAULT
```

Quellenhinweise

Das Skript wurde aus den bereitgestellten Unterlagen M0 bis M10, dem SQL-Merkblatt, der Datei *M_Klausurvorbereitung_ANGABE_LÖSUNG_V2.1*, der Altklausur vom 28. Juni 2021 und der Klausur vom 7. Oktober 2021 zusammengestellt. Eingebettete Folien sind jeweils mit Quelldatei und Foliennummer beschriftet. Die Übungsaufgaben sind teilweise gekürzt oder didaktisch angepasst; maßgeblich für Originalformulierungen sind die bereitgestellten PDFs.